



LoanHub v1.0

Technical Architecture, Source-Code Map and Developer Handover

Release	LoanHub v1.0
Audience	Software developers, DevOps, database administrators and technical support
Prepared	16 July 2026
Developer	Ithute Solutions



Developed and maintained by Ithute Solutions



Document control

Item	Value
Product	LoanHub multi-tenant loan marketplace and management platform
Release	v1.0
Purpose	Provide the next programmer with enough architecture, code and workflow knowledge to maintain and upgrade v1.0 safely.
Audience	Developers, architects, DevOps and database administrators
Developer	Ithute Solutions
Source baseline	Final engineered monorepo plus WebSocket, file-upload and accounting race fixes
Database head	f1a9c4e7b620
Business timezone	Africa/Maseru

Version note

The product handover is named LoanHub v1.0. The FastAPI metadata string in the current source is an independent API schema version and should be governed consistently in future releases.

Contents

1. Executive technical summary
2. Architecture and repository map
3. Technology stack
4. Frontend architecture
5. Backend architecture
6. Authentication, tenancy and roles
7. Database models and migrations
8. API catalog
9. Business workflows
10. Realtime chat and presence
11. Managed files and encryption



LoanHub v1.0 Technical Architecture and Developer Handover

12. Notifications, audit and error monitoring

13. Accounting and reporting

14. Midnight automation

15. Docker, CI/CD and Hostinger deployment

16. Configuration and secrets

17. Code-change playbook

18. Testing and release validation

19. Troubleshooting

20. Known boundaries and v1.1 backlog

21. Appendix catalogs



1. Executive technical summary

LoanHub v1.0 is a modular-monolith SaaS platform. The Next.js frontend uses typed API clients, Redux and context providers. FastAPI exposes REST and WebSocket endpoints. SQLAlchemy and PostgreSQL implement tenant-scoped business transactions; Redis distributes realtime events across workers; a maintenance process performs reconciliation and scheduled reporting. Docker Compose packages the full runtime behind Caddy TLS.

Metric	v1.0 snapshot
Backend route operations	146
Mapped SQLAlchemy models	37
Next.js pages	51
Alembic revisions	34, one head f1a9c4e7b620
Backend Python files	149
Frontend TypeScript/TSX files	267

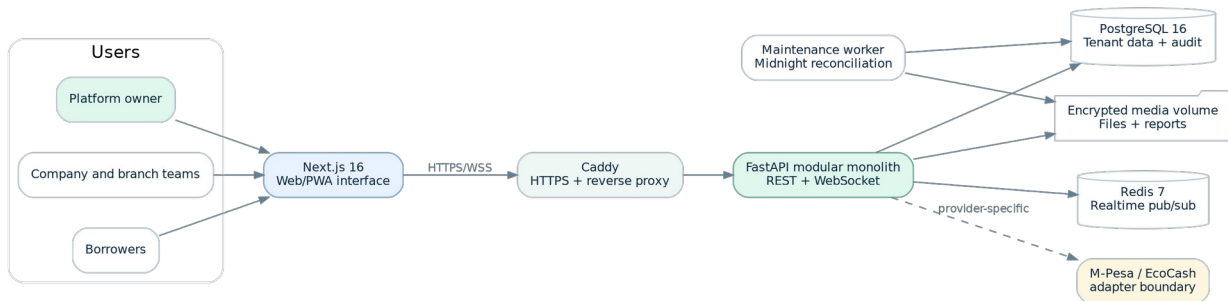


Figure 1. Runtime context and service boundaries.

2. Architecture and repository map

Repository structure

```
LoanHub/
├── apps/
│   ├── backend/
│   │   ├── api/v1/router.py
│   │   ├── routers/
│   │   ├── database/models/
│   │   ├── database/schemas/
│   │   ├── services/
│   │   ├── core/
│   │   ├── alembic/versions/
│   │   └── docker/maintenance.py
│   └── frontend/
│       ├── app/
│       ├── components/
│       └── provider/
```



```
├── store/
├── api/
├── types/
├── lib/ and utils/
├── brand/
├── docs/
├── infra/caddy/
├── scripts/
├── secrets/
├── compose.yaml
└── .github/workflows/deploy-hostinger.yml
```

Modular-monolith rationale

- Tenant authorisation and financial changes remain inside a single database transaction.
- Routers, schemas, services and models provide clear internal boundaries.
- Domains can later be extracted only after interfaces, idempotency and event delivery are stable.
- Redis already removes the single-process limitation from realtime delivery.

3. Technology stack

Layer	Technology	Purpose
Frontend	Next.js 16.2.4, React 19.2.4, TypeScript 5	Routing, rendering, typed UI
UI	Tailwind CSS 4, shadcn/Radix, Lucide	Responsive components and themes
State	Redux Toolkit, React providers, Axios	Auth, tenant data, realtime and API state
Backend	FastAPI, Pydantic, SQLAlchemy	REST/WebSocket and business logic
Database	PostgreSQL 16, Alembic	Transactional data and schema history
Realtime	WebSocket + Redis 7 pub/sub	Cross-worker notifications, chat and presence
Files	Persistent media volume + AES-GCM service	Managed files and reports
Proxy	Caddy 2	TLS and reverse proxy
Automation	GitHub Actions, Docker Compose	Validation and deployment



4. Frontend architecture

State and provider order

Authentication owns the user/token. Tenant selection owns the active company. Realtime owns the single WebSocket. Notification and Chat providers subscribe to that shared connection. AppData then loads role-specific collections and analytics.

Provider composition

```
<AuthProvider>
  <TenantProvider>
    <RealtimeProvider>
      <NotificationProvider>
        <ChatProvider>
          <AppDataProvider>{children}</AppDataProvider>
        </ChatProvider>
      </NotificationProvider>
    </RealtimeProvider>
  </TenantProvider>
</AuthProvider>
```

Axios request and refresh behaviour

`apps/frontend/lib/api.ts` - token, tenant header and FormData handling

```
export const api = axios.create({
  baseURL:
    process.env.NEXT_PUBLIC_API_URL ??
    "http://localhost:8000/api/v1",
  withCredentials: true,
});

type RetryableRequest = InternalAxiosRequestConfig & {
  _retry?: boolean;
};

let refreshPromise: Promise<string> | null = null;

function restoreOriginalPlatformSession(): boolean {
  const impersonation = getStoredValue(StorageKeys.impersonation);
  const originalToken = getStoredValue(StorageKeys.originalAdminToken);
  const originalUser = getStoredJson<AuthUser>(StorageKeys.originalAdminUser);

  if (!impersonation || !originalToken || !originalUser) {
    return false;
  }

  setStoredValue(StorageKeys.accessToken, originalToken);
  setStoredValue(StorageKeys.authUser, JSON.stringify(originalUser));
  removeStoredValue(StorageKeys.originalAdminToken);
  removeStoredValue(StorageKeys.originalAdminUser);
  removeStoredValue(StorageKeys.impersonation);
  removeStoredValue(StorageKeys.activeCompanyId);

  if (typeof window !== "undefined") {
    window.location.assign("/superadmin?reason=impersonation-ended");
  }

  return true;
}

function refreshAccessToken(): Promise<string> {
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
if (!refreshPromise) {
  refreshPromise = api
    .post<RefreshResponse>("/auth/refresh")
    .then((response) => {
      const token = response.data.access_token;
      setStoredValue(StorageKeys.accessToken, token);
      return token;
    })
    .finally(() => {
      refreshPromise = null;
    });
}
return refreshPromise;
}

api.interceptors.request.use((config: InternalAxiosRequestConfig) => {
  const token = getStoredValue(StorageKeys.accessToken);
  const activeCompanyId = getStoredValue(StorageKeys.activeCompanyId);
  # ... remaining implementation omitted in this guide
}
```

Page routes

Route	Source page
/forgot-password	(auth)/forgot-password/page.tsx
/login	(auth)/login/page.tsx
/borrower/chat	(dashboard)/borrower/chat/page.tsx
/borrower/files	(dashboard)/borrower/files/page.tsx
/borrower/loans	(dashboard)/borrower/loans/page.tsx
/borrower/notifications	(dashboard)/borrower/notifications/page.tsx
/borrower	(dashboard)/borrower/page.tsx
/borrower/payments	(dashboard)/borrower/payments/page.tsx
/borrower/profile	(dashboard)/borrower/profile/page.tsx
/borrower/requests	(dashboard)/borrower/requests/page.tsx
/company/accounting	(dashboard)/company/accounting/page.tsx
/company/activity	(dashboard)/company/activity/page.tsx
/company/billing	(dashboard)/company/billing/page.tsx
/company/branches	(dashboard)/company/branches/page.tsx
/company/chat	(dashboard)/company/chat/page.tsx
/company/employees	(dashboard)/company/employees/page.tsx
/company/files	(dashboard)/company/files/page.tsx
/company/loans	(dashboard)/company/loans/page.tsx



LoanHub v1.0 Technical Architecture and Developer Handover

Route	Source page
/company/marketplace	(dashboard)/company/marketplace/page.tsx
/company/notifications	(dashboard)/company/notifications/page.tsx
/company	(dashboard)/company/page.tsx
/company/payments	(dashboard)/company/payments/page.tsx
/company/performance	(dashboard)/company/performance/page.tsx
/company/products	(dashboard)/company/products/page.tsx
/company/reports	(dashboard)/company/reports/page.tsx
/company/settings	(dashboard)/company/settings/page.tsx
/company/staff	(dashboard)/company/staff/page.tsx
/company-admin	(dashboard)/company-admin/page.tsx
/superadmin/accounting	(dashboard)/superadmin/accounting/page.tsx
/superadmin/activity	(dashboard)/superadmin/activity/page.tsx
/superadmin/chat	(dashboard)/superadmin/chat/page.tsx
/superadmin/companies/[id]	(dashboard)/superadmin/companies/[id]/page.tsx
/superadmin/companies/branches	(dashboard)/superadmin/companies/branches/page.tsx
/superadmin/companies	(dashboard)/superadmin/companies/page.tsx
/superadmin/company-admins	(dashboard)/superadmin/company-admins/page.tsx
/superadmin/files	(dashboard)/superadmin/files/page.tsx
/superadmin/loans/[id]	(dashboard)/superadmin/loans/[id]/page.tsx
/superadmin/loans	(dashboard)/superadmin/loans/page.tsx
/superadmin/notifications	(dashboard)/superadmin/notifications/page.tsx
/superadmin	(dashboard)/superadmin/page.tsx
/superadmin/payments	(dashboard)/superadmin/payments/page.tsx
/superadmin/performance	(dashboard)/superadmin/performance/page.tsx
/superadmin/plans	(dashboard)/superadmin/plans/page.tsx
/superadmin/reports	(dashboard)/superadmin/reports/page.tsx
/superadmin/system-errors	(dashboard)/superadmin/system-errors/page.tsx
/borrower-registration	(visitors)/borrower-registration/page.tsx



LoanHub v1.0 Technical Architecture and Developer Handover

Route	Source page
/choose-account-type	(visitors)/choose-account-type/page.tsx
/greetings	(visitors)/greetings/page.tsx
/lender-access	(visitors)/lender-access/page.tsx
/register-company-admin	(visitors)/register-company-admin/page.tsx
/	page.tsx

Where to change frontend features

Change	Files to inspect first
Authentication/session	`api/auth.ts`, `lib/api.ts`, auth provider and auth slice
Tenant selection	tenant provider, storage helpers, Axios interceptor
Realtime	`provider/realtimeProvider.tsx`; do not create another socket
Notifications/chat	notification/chat providers and corresponding API clients
Role navigation	role shells, navigation config, `lib/role-redirect.ts`
Page workflow	matching `app` page, reusable component, API client and types
Themes	theme provider, theme switcher and global CSS
API errors/toasts	`utils/apiError.ts` and `utils/toast.ts`



LoanHub v1.0 Technical Architecture and Developer Handover

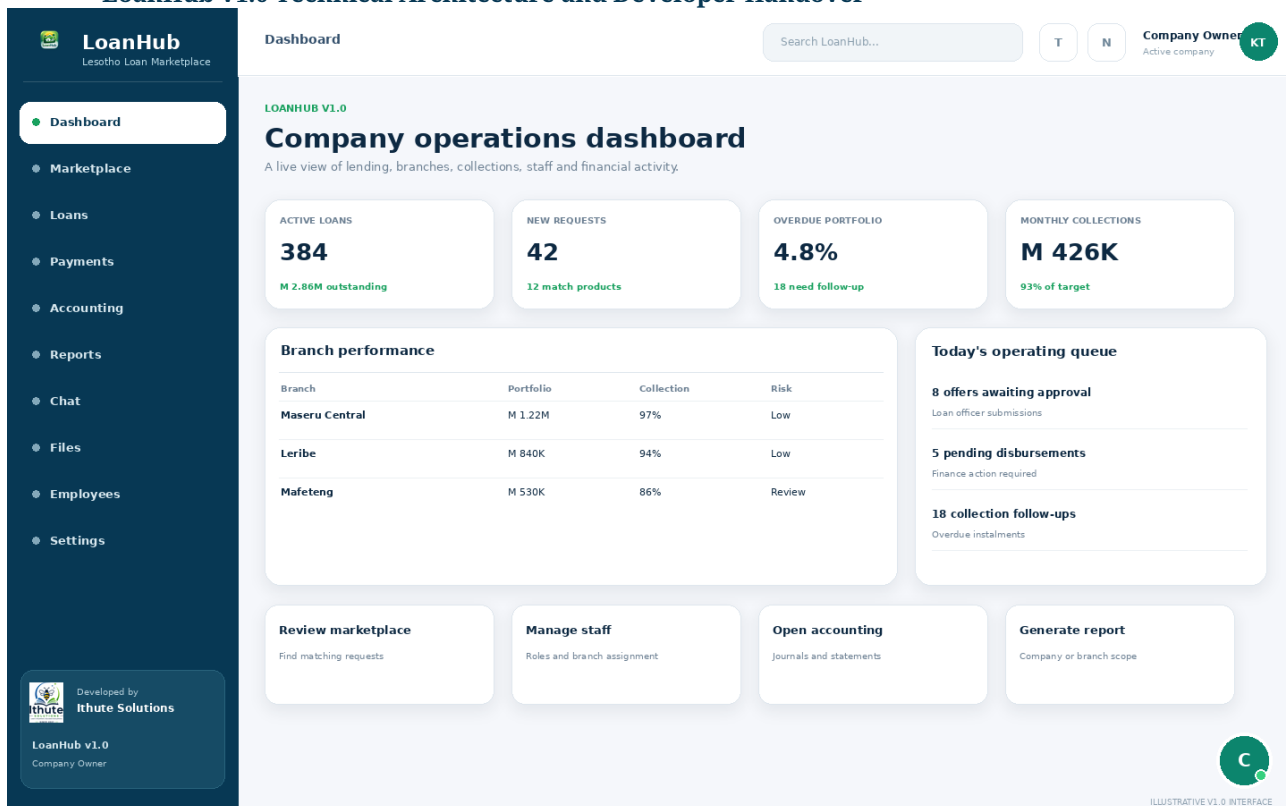


Figure 2. Implemented company-dashboard design language.

5. Backend architecture

Application composition

`apps/backend/main.py` - middleware, router and realtime startup

```
from fastapi import Depends, FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import text
from sqlalchemy.orm import Session

from api.v1.router import api_router
import core.transparency # noqa: F401 - registers SQLAlchemy transparency listeners
from core.error_monitoring import SystemErrorMonitoringMiddleware
from core.websocket_manager import manager
from database.config.config import settings
from database.session import get_db
from utils.authContextMiddleware import AuthContextMiddleware

app = FastAPI(
    title="LoanHub API",
    version="2.0.0",
    description=(
        "Secure multi-tenant loan marketplace, management, accounting, "
        "reporting, files and realtime communication platform"
    ),
)

app.add_middleware(SystemErrorMonitoringMiddleware)
app.add_middleware(AuthContextMiddleware)

app.add_middleware(
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
CORSMiddleware,
allow_origins=settings.cors_origins,
allow_credentials=True,
allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
allow_headers=["Authorization", "Content-Type", "Accept", "X-Company-ID", "X-Request-ID"],
)

app.include_router(api_router, prefix="/api/v1")

@app.get("/", tags=["System"])
def root():
    return {
        "message": "LoanHub API is running",
        "environment": settings.ENVIRONMENT,
    }

@app.get("/health", include_in_schema=False)
def health(db: Session = Depends(get_db)):
    db.execute(text("SELECT 1"))
    return {"status": "healthy"}

@app.on_event("startup")
async def start_realtime_bridge() -> None:
    await manager.start()

@app.on_event("shutdown")
async def stop_realtime_bridge() -> None:
    await manager.shutdown()
```

Router registration

`api/v1/router.py` - domain routers

```
from fastapi import APIRouter

from routers import (
    reports,
    files,
    chat,
    accounting,
    audit,
    auth,
    billing,
    borrower,
    borrower_registration,
    employees,
    branches,
    company,
    company_registration,
    company_staff,
    loan_offers,
    loan_products,
    loan_request,
    loans,
    marketplace,
    payments,
    performance,
    person,
    notifications,
    system_errors,
    ws,
)

api_router = APIRouter()

api_router.include_router(auth.router)
api_router.include_router(notifications.router)
```



```
api_router.include_router(audit.router)
api_router.include_router(company_registration.router)
api_router.include_router(borrower_registration.router)
```

```
api_router.include_router(company.router)
api_router.include_router(company_staff.router)
api_router.include_router(employees.router)
api_router.include_router(performance.router)
api_router.include_router(branches.router)
api_router.include_router(person.router)
api_router.include_router(borrower.router)
```

```
api_router.include_router(loan_request.router)
api_router.include_router(marketplace.router)
api_router.include_router(loan_offers.router)
api_router.include_router(loan_products.router)
api_router.include_router(loans.router)
api_router.include_router(payments.router)
api_router.include_router(billing.router)
api_router.include_router(system_errors.router)
```

```
api_router.include_router(files.router)
```

```
api_router.include_router(chat.router)
```

```
api_router.include_router(accounting.router)
```

```
api_router.include_router(reports.router)
```

```
api_router.include_router(ws.router)
```

Change pattern

Most features should follow Router -> Pydantic Schema -> Service -> SQLAlchemy Model. A model change requires an Alembic migration. Cross-cutting security and audit behaviour belongs in core modules rather than individual pages.

6. Authentication, tenancy and roles

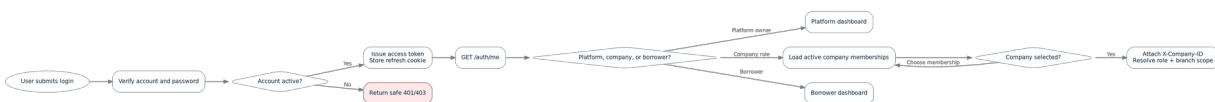


Figure 3. Login, role routing and tenant-context resolution.

`core/access\_control.py` - active user, platform permission and tenant/branch scope

```
def get_current_active_user(
    current_user: User = Depends(get_current_user),
) -> User:
    if not current_user.is_active:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Account is inactive",
        )
    return current_user

def require_platform_admin(
    current_user: User = Depends(get_current_active_user),
) -> User:
    if current_user.role != UserRole.SUPERADMIN:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Platform administrator permission is required",
        )
    return current_user
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
)
return current_user

def _parse_company_header(value: str | None) -> UUID | None:
    if not value:
        return None
    try:
        return UUID(value)
    except ValueError as error:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="X-Company-ID must be a valid UUID",
        ) from error

def resolve_tenant_context(
    db: Session,
    current_user: User,
    x_company_id: str | None = None,
) -> TenantContext:
    if current_user.role == UserRole.SUPERADMIN:
        return TenantContext(
            user=current_user,
            staff=None,
            company=None,
            company_id=None,
            branch_id=None,
            is_platform_admin=True,
        )

    requested_company_id = _parse_company_header(x_company_id)

    query = (
        db.query(CompanyStaff)
        .options(joinedload(CompanyStaff.company))
        .filter(
            CompanyStaff.user_id == current_user.id,
            CompanyStaff.is_active.is_(True),
        )
    )

    if requested_company_id:
        query = query.filter(CompanyStaff.company_id == requested_company_id)

    memberships = query.order_by(CompanyStaff.created_at.asc()).all()

    if not memberships:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="No active loan-company membership was found for this account",
        )

    if not requested_company_id and len(memberships) > 1:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=(
                "This account belongs to multiple companies. "
                "Send the selected company in the X-Company-ID header."
            ),
        )

    staff = memberships[0]
    company = staff.company

    if not company:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="The staff membership is not linked to a company",
        )

    if company.status != CompanyStatus.APPROVED or not company.is_active:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="The selected company is not approved and active",
        )
)
```



```

return TenantContext(
    user=current_user,
    staff=staff,
    company=company,
    company_id=staff.company_id,
    branch_id=staff.branch_id,
    is_platform_admin=False,
)

def get_tenant_context(
    x_company_id: str | None = Header(default=None, alias="X-Company-ID"),
    current_user: User = Depends(get_current_active_user),
    db: Session = Depends(get_db),
) -> TenantContext:
    return resolve_tenant_context(db, current_user, x_company_id)

def require_tenant_roles(
    context: TenantContext,
    allowed_roles: Iterable[UserRole],
) -> None:
    if context.is_platform_admin:
        return

    allowed = set(allowed_roles)
    if not context.staff or context.staff.role not in allowed:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You do not have permission to perform this company operation",
        )

def assert_company_scope(
    context: TenantContext,
    company_id: UUID,
) -> None:
    if context.is_platform_admin:
        return
    if context.company_id != company_id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Cross-company access is not allowed",
        )

def assert_branch_scope(
    context: TenantContext,
    branch_id: UUID | None,
) -> None:
    if context.is_platform_admin or not context.staff:
        return

    # Company owners and company administrators have company-wide visibility.
    if context.staff.role in COMPANY_MANAGEMENT_ROLES:
        return

    # Other staff accounts become branch-scoped whenever their membership has
    # a branch assignment. This prevents a user from guessing an ID belonging
    # to another branch even when both branches belong to the same company.
    if context.branch_id is not None and context.branch_id != branch_id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="This record belongs to another company branch",
        )

```

Role impersonation





- Only the platform owner can start a session.
- The target must be an eligible active user/membership.
- The reason and duration are recorded.
- The original owner token/user are preserved and restored.
- An impersonation banner must remain visible.
- The impersonated token is short-lived and must not grant more than the target role.

7. Database models and migrations

The source snapshot maps 37 SQLAlchemy models. Common audit fields are inherited from the shared base. The table catalog below gives the programmer a first location for persistence changes; the CSV pack includes columns, relationships and foreign keys.

Table	Model	Purpose	Foreign-key summary
accounting_accounts	AccountingAccount	Chart of accounts.	company_id->loan_companies.id, branch_id->company_branches.id, parent_id->accounting_accounts.id
audit_logs	AuditLog	Who changed what, where and when.	user_id->users.id, company_id-> loan_companies.id, branch_id-> company_branches.id
borrower_documents	BorrowerDocument	Borrower identity and supporting evidence.	borrower_id->borrowers.id
borrowers	Borrower	Borrower profile and affordability context.	user_id->users.id
broadcast	Broadcast	Domain persistence record.	user_id->users.id
chat_conversations	ChatConversation	Direct/group chat container.	company_id->loan_companies.id, branch_id->company_branches.id, created_by_user_id->users.id
chat_message_attachments	ChatMessageAttachment	Links messages to managed files.	message_id->chat_messages.id, file_id-> managed_files.id
chat_messages	ChatMessage	Encrypted/managed chat message records.	conversation_id->chat_conversations.id, sender_user_id->users.id, company_id-> loan_companies.id, branch_id-> company_branches.id, reply_to_message_id->chat_messages.id
chat_participants	ChatParticipant	Conversation membership and read state.	conversation_id->chat_conversations.id, user_id->users.id
client_company_loan	ClientCompanyLoan	Domain persistence record.	loan_request_id->loan_requests.id, loan_offer_id->loan_offers.id, company_id->loan_companies.id, branch_id->company_branches.id, borrower_id->borrowers.id, approved_by_user_id->users.id, disbursed_by_user_id->users.id
company_branches	CompanyBranch	Domain persistence record.	company_id->loan_companies.id
company_staff	CompanyStaff	User membership, company role and branch scope.	user_id->users.id, company_id-> loan_companies.id, branch_id-



LoanHub v1.0 Technical Architecture and Developer Handover

Table	Model	Purpose	Foreign-key summary
			>company_branches.id
company_subscriptions	CompanySubscription	Company plan lifecycle.	company_id->loan_companies.id, plan_id->subscription_plans.id
employee_profiles	EmployeeProfile	Domain persistence record.	staff_id->company_staff.id, company_id-> loan_companies.id, branch_id-> company_branches.id, reports_to_staff_id->company_staff.id
generated_reports	GeneratedReport	Generated report metadata and managed-file link.	company_id->loan_companies.id, branch_id->company_branches.id, schedule_id->report_schedules.id, generated_by_user_id->users.id, file_id-> managed_files.id
journal_entries	JournalEntry	Accounting transaction headers.	company_id->loan_companies.id, branch_id->company_branches.id, created_by_user_id->users.id, posted_by_user_id->users.id
journal_lines	JournalLine	Balanced debit/credit lines.	journal_entry_id->journal_entries.id, account_id->accounting_accounts.id
lender_access_requests	LenderAccessRequest	Legacy/controlled lender access request.	loan_request_id->loan_requests.id, company_id->loan_companies.id, requested_by_user_id->users.id
loan_companies	LoanCompany	Domain persistence record.	-
loan_offers	LoanOffer	Company offers made against requests.	loan_request_id->loan_requests.id, company_id->loan_companies.id, branch_id->company_branches.id, offered_by_user_id->users.id
loan_products	LoanProduct	Company-controlled loan products and pricing bounds.	company_id->loan_companies.id
loan_request_documents	LoanRequestDocument	Domain persistence record.	loan_request_id->loan_requests.id
loan_requests	LoanRequest	Borrower marketplace funding requests.	borrower_id->borrowers.id, selected_offer_id->loan_offers.id
managed_files	ManagedFile	File metadata, visibility, integrity and encryption.	owner_user_id->users.id, company_id-> loan_companies.id, branch_id-> company_branches.id
marketplace_unlocks	MarketplaceUnlock	Company access/unlock events for requests.	company_id->loan_companies.id, loan_request_id->loan_requests.id, payment_transaction_id-> payment_transactions.id, unlocked_by_user_id->users.id
notifications	Notification	Persistent user-targeted actionable events.	user_id->users.id, actor_user_id-> users.id, company_id-> loan_companies.id, branch_id-> company_branches.id, related_loan_request_id-> loan_requests.id, related_offer_id-> loan_offers.id
payment_allocations	PaymentAllocation	Allocation of repayments to instalments.	payment_id->payment_transactions.id, installment_id-> repayment_installments.id



LoanHub v1.0 Technical Architecture and Developer Handover

Table	Model	Purpose	Foreign-key summary
payment_transactions	PaymentTransaction	Provider-neutral inbound/outbound payment ledger.	company_id->loan_companies.id, borrower_id->borrowers.id, loan_request_id->loan_requests.id, loan_id->client_company_loan.id, initiated_by_user_id->users.id
people	Person	Reusable person identity/profile data.	user_id->users.id
performance_goals	PerformanceGoal	Targets and progress.	employee_id->employee_profiles.id, company_id->loan_companies.id, branch_id->company_branches.id, created_by_user_id->users.id
performance_reviews	PerformanceReview	Review-period assessment data.	employee_id->employee_profiles.id, reviewer_staff_id->company_staff.id, company_id->loan_companies.id, branch_id->company_branches.id
refresh_tokens	RefreshToken	Refresh-token lifecycle and revocation.	user_id->users.id
repayment_installments	RepaymentInstallment	Scheduled instalments and due/paid status.	loan_id->client_company_loan.id
report_schedules	ReportSchedule	Recurring report configuration.	company_id->loan_companies.id, branch_id->company_branches.id, created_by_user_id->users.id
subscription_plans	SubscriptionPlan	Platform commercial plan configuration.	-
system_error_logs	SystemErrorLog	Grouped technical failures and resolution status.	user_id->users.id, company_id->loan_companies.id, branch_id->company_branches.id, resolved_by_user_id->users.id
users	User	Authentication accounts, global role, status and presence.	-

Migration discipline

Rule	Reason
Never delete an applied revision	The database stores the revision identifier and future Alembic commands require the file.
Do not use `stamp head` to hide a mismatch	Stamping does not apply schema changes.
Stop API/worker before risky DDL	Avoid locks and migration deadlocks.
Review autogenerate output	Model naming collisions can appear as destructive changes.
Keep one head	Deployment and rollback become deterministic.
Back up first	Financial and document metadata must be recoverable.



LoanHub v1.0 Technical Architecture and Developer Handover

#	Revision	Down revision	Purpose/title
1	3057a2cdb0ea		initial loan marketplace schema
2	4ffb06dcde89	3057a2cdb0ea	initial loan marketplace schema
3	b0a8b024a79e	4ffb06dcde89	initial loan marketplace schema
4	f86a6ab5a269	b0a8b024a79e	initial loan marketplace schema
5	96e6e94f2ba7	f86a6ab5a269	c
6	790ed6fd651d	96e6e94f2ba7	c
7	15c00a058229	790ed6fd651d	c
8	d09fde6cd60e	15c00a058229	c
9	8563cd58deb9	d09fde6cd60e	c
10	464f4729f8a7	8563cd58deb9	c
11	920a67b52323	464f4729f8a7	c
12	db8c1b7032c2	920a67b52323	c
13	77031f1f0be4	db8c1b7032c2	c
14	b0c4c142919a	77031f1f0be4	c
15	48e0a5d6cb1f	b0c4c142919a	c
16	d16e137105d7	48e0a5d6cb1f	add school admin
17	44822a754848	d16e137105d7	add school admin
18	80d0284b8cd6	44822a754848	add loan company client
19	a56c9061c634	80d0284b8cd6	add loan company client
20	cdb7c1666d74	a56c9061c634	add loan company client
21	056d9353ae4b	cdb7c1666d74	add loan company client
22	af6a1575fb01	056d9353ae4b	add loan company client
23	fd4aa8a11b41	af6a1575fb01	add person profiles
24	7ba90e8af6b6	fd4aa8a11b41	add people model
25	9f1c2d3e4a5b	7ba90e8af6b6	multitenant SaaS, billing, payments and repayment foundation
26	307dcdff0187c	9f1c2d3e4a5b	normalize company staff company cascade
27	46b05ea64319	307dcdff0187c	expand payment idempotency key
28	274950a4cd45	46b05ea64319	legacy migration-chain compatibility



LoanHub v1.0 Technical Architecture and Developer Handover

#	Revision	Down revision	Purpose/title
			marker
29	6c1f9a7e2d40	274950a4cd45	realtime transparency, employees, performance and system errors
30	3957062b7c66	6c1f9a7e2d40	compatibility marker for an already-recorded local revision
31	8d2f4a1b7c90	3957062b7c66	marketable chat, files, reports and accounting
32	b84d1f2a9c30	8d2f4a1b7c90	canonical schema repair after compatibility and marketable revisions
33	d4e7b6c1a930	b84d1f2a9c30	align legacy index names and restore the selected-offer foreign key
34	f1a9c4e7b620	d4e7b6c1a930	final security, presence and encrypted content alignment

8. Complete API catalog

All domain operations are included below. Access hints are inferred from route dependencies and should be confirmed in the function body before modification.

accounting

Method	Path	Function	Access hint
POST	/accounting/bootstrap	bootstrap_chart	route-specific
GET	/accounting/accounts	list_accounts	route-specific
POST	/accounting/accounts	create_account	route-specific
PATCH	/accounting/accounts/{account_id}	update_account	route-specific
GET	/accounting/journal-entries	list_entries	route-specific
POST	/accounting/journal-entries	add_entry	route-specific
POST	/accounting/journal-entries/{entry_id}/post	post_entry	route-specific
GET	/accounting/trial-balance	trial_balance	route-specific
GET	/accounting/profit-and-loss	profit_and_loss	route-specific
GET	/accounting/balance-sheet	balance_sheet	route-specific



audit

Method	Path	Function	Access hint
GET	/audit/events	list_audit_events	authenticated, tenant context

auth

Method	Path	Function	Access hint
POST	/auth/login	login	route-specific
POST	/auth/websocket-session	create_websocket_session	authenticated
POST	/auth/password-reset-request	request_password_reset	route-specific
GET	/auth/me	me	authenticated
POST	/auth/refresh	refresh_token	route-specific
POST	/auth/logout	logout	platform owner
GET	/auth/users	list_users	platform owner
GET	/auth/users/{user_id}	get_user	platform owner
PUT	/auth/users/{user_id}	update_user	platform owner
DELETE	/auth/users/{user_id}	delete_user	platform owner
GET	/auth/impersonation-targets	impersonation_targets	platform owner
POST	/auth/impersonate	impersonate_user	platform owner

billing

Method	Path	Function	Access hint
GET	/billing/plans	list_public_plans	platform owner
GET	/billing/plans/admin	list_all_plans_for_admin	platform owner
POST	/billing/plans	create_plan	platform owner
PUT	/billing/plans/{plan_id}	update_plan	platform owner
DELETE	/billing/plans/{plan_id}	delete_plan	platform owner
GET	/billing/subscriptions	list_subscriptions	route-specific
GET	/billing/subscriptions/current	current_subscription	route-specific



LoanHub v1.0 Technical Architecture and Developer Handover

Method	Path	Function	Access hint
POST	/billing/subscriptions/checkout	subscription_checkout	route-specific
POST	/billing/subscriptions/{subscription_id}/cancel	cancel_subscription	route-specific

borrower

Method	Path	Function	Access hint
GET	/borrowers/	list_borrowers	authenticated, platform owner
GET	/borrowers/me	get_my_borrower_profile	authenticated, platform owner
GET	/borrowers/{borrower_id}	get_borrower	authenticated, platform owner
POST	/borrowers/	create_borrower	authenticated, platform owner
PUT	/borrowers/{borrower_id}	update_borrower	authenticated, platform owner
DELETE	/borrowers/{borrower_id}	delete_borrower	platform owner

borrower_registration

Method	Path	Function	Access hint
POST	/borrower-registration/	register_borrower	route-specific

branches

Method	Path	Function	Access hint
GET	/branches/	list_branches	route-specific
GET	/branches/company/{company_id}	list_company_branches	route-specific
GET	/branches/{branch_id}	get_branch	route-specific
POST	/branches/	create_branch	route-specific
PUT	/branches/{branch_id}	update_branch	route-specific
DELETE	/branches/{branch_id}	delete_branch	route-specific
PATCH	/branches/{branch_id}/activate	activate_branch	route-specific
PATCH	/branches/{branch_id}/deactivate	deactivate_branch	route-specific



chat

Method	Path	Function	Access hint
GET	/chat/directory	chat_directory	route-specific
GET	/chat/conversations	list_conversations	route-specific
GET	/chat/unread-count	unread_count	route-specific
POST	/chat/conversations	create_conversation	route-specific
GET	/chat/conversations/{conversation_id}/messages	list_messages	route-specific
POST	/chat/conversations/{conversation_id}/messages	send_message	route-specific
POST	/chat/conversations/{conversation_id}/files	send_file_message	route-specific
PATCH	/chat/conversations/{conversation_id}/read	mark_conversation_read	route-specific
PATCH	/chat/messages/{message_id}	update_message	route-specific
DELETE	/chat/messages/{message_id}	delete_message	route-specific

company

Method	Path	Function	Access hint
POST	/companies/	create_company	authenticated, platform owner
GET	/companies/	list_companies	authenticated
GET	/companies/{company_id}	get_company	authenticated
PUT	/companies/{company_id}	update_company	authenticated, platform owner
DELETE	/companies/{company_id}	delete_company	platform owner
PATCH	/companies/{company_id}/approve	approve_company	platform owner
PATCH	/companies/{company_id}/reject	reject_company	platform owner
PATCH	/companies/{company_id}/activate	activate_company	platform owner
PATCH	/companies/{company_id}/deactivate	deactivate_company	platform owner
PATCH	/companies/{company_id}/suspend	suspend_company	platform owner



company_registration

Method	Path	Function	Access hint
POST	/company-registration/	register_company_owner	route-specific

company_staff

Method	Path	Function	Access hint
GET	/company-staff/	list_company_staff	route-specific
GET	/company-staff/{staff_id}	get_company_staff_member	route-specific
POST	/company-staff/	assign_existing_user	route-specific
POST	/company-staff/accounts	create_staff_account	route-specific
PUT	/company-staff/{staff_id}	update_company_staff	route-specific
DELETE	/company-staff/{staff_id}	remove_company_staff	route-specific

employees

Method	Path	Function	Access hint
GET	/employees	list_employees	route-specific
GET	/employees/{staff_id}	get_employee	route-specific
PUT	/employees/{staff_id}/profile	upsert_employee_profile	route-specific
GET	/employees/{staff_id}/goals	list_employee_goals	route-specific
POST	/employees/{staff_id}/goals	create_employee_goal	route-specific
PATCH	/employees/goals/{goal_id}	update_goal	route-specific
GET	/employees/{staff_id}/reviews	list_employee_reviews	route-specific
POST	/employees/{staff_id}/reviews	create_employee_review	route-specific

files

Method	Path	Function	Access hint
GET	/files	list_files	route-specific
POST	/files/upload	upload_file	route-specific



LoanHub v1.0 Technical Architecture and Developer Handover

Method	Path	Function	Access hint
GET	/files/{file_id}	get_file	route-specific
GET	/files/{file_id}/download	download_file	route-specific
DELETE	/files/{file_id}	delete_file	route-specific

loan_offers

Method	Path	Function	Access hint
POST	/loan-offers/	create_offer	route-specific
GET	/loan-offers/request/{loan_request_id}	list_offers_by_request	authenticated, tenant context
GET	/loan-offers/{offer_id}	get_offer	authenticated, tenant context
PATCH	/loan-offers/{offer_id}	update_offer	route-specific
POST	/loan-offers/{offer_id}/withdraw	withdraw_offer	route-specific
DELETE	/loan-offers/{offer_id}	delete_offer	route-specific

loan_products

Method	Path	Function	Access hint
GET	/loan-products/public	list_public_products	route-specific
GET	/loan-products/	list_products	route-specific
GET	/loan-products/{product_id}	get_product	route-specific
POST	/loan-products/	create_product	route-specific
PUT	/loan-products/{product_id}	update_product	route-specific
PATCH	/loan-products/{product_id}/activate	activate_product	route-specific
PATCH	/loan-products/{product_id}/deactivate	deactivate_product	route-specific
DELETE	/loan-products/{product_id}	delete_product	route-specific

loan_request

Method	Path	Function	Access hint
POST	/loan_requests/	create_request	authenticated, borrower, platform



LoanHub v1.0 Technical Architecture and Developer Handover

Method	Path	Function	Access hint
			owner
GET	/loan_requests/	get_all_requests	authenticated, borrower, platform owner
GET	/loan_requests/my	get_my_requests	authenticated, borrower, platform owner
GET	/loan_requests/open	get_open_requests	authenticated, borrower, platform owner
GET	/loan_requests/{request_id}	get_request	authenticated, borrower
PATCH	/loan_requests/{request_id}	update_request	authenticated, borrower
POST	/loan_requests/{request_id}/submit	submit_request	authenticated, borrower
POST	/loan_requests/{request_id}/cancel	cancel_request	authenticated, borrower
POST	/loan_requests/{request_id}/offers/{offer_id}/accept	accept_request_offer	authenticated, borrower
DELETE	/loan_requests/{request_id}	delete_request	authenticated, borrower

loans

Method	Path	Function	Access hint
GET	/loans/	list_loans	authenticated, tenant context
GET	/loans/{loan_id}	get_loan	authenticated, tenant context
POST	/loans/{loan_id}/disburse	disburse_loan	authenticated
POST	/loans/{loan_id}/repay	repay_loan	authenticated

marketplace

Method	Path	Function	Access hint
GET	/marketplace/requests	list_marketplace_requests	route-specific
GET	/marketplace/requests/{request_id}	get_marketplace_request	route-specific
POST	/marketplace/requests/{request_id}/unlock	unlock_request	route-specific



notifications

Method	Path	Function	Access hint
GET	/notifications	list_notifications	authenticated
GET	/notifications/unread-count	unread_count	authenticated
PATCH	/notifications/{notification_id}/read	mark_notification_read	authenticated
PATCH	/notifications/read-all	mark_all_read	authenticated
PATCH	/notifications/{notification_id}/archive	archive_notification	authenticated
DELETE	/notifications/{notification_id}	delete_notification	authenticated

payments

Method	Path	Function	Access hint
GET	/payments/	list_payments	authenticated, tenant context
GET	/payments/{payment_id}	get_payment	authenticated, tenant context
POST	/payments/callbacks/{provider}	payment_callback	route-specific

performance

Method	Path	Function	Access hint
GET	/performance/overview	performance_overview	route-specific

person

Method	Path	Function	Access hint
GET	/people/	list_people	authenticated, platform owner
GET	/people/me	get_my_person	authenticated
GET	/people/user/{user_id}	get_person_by_user_id	authenticated
GET	/people/{person_id}	get_person	authenticated
POST	/people/	create_person	authenticated
PUT	/people/{person_id}	update_person	authenticated, platform owner
DELETE	/people/{person_id}	delete_person	platform owner



reports

Method	Path	Function	Access hint
GET	/reports/summary	report_summary	route-specific
GET	/reports	list_reports	route-specific
POST	/reports/generate	generate_now	route-specific
GET	/reports/schedules	list_schedules	route-specific
POST	/reports/schedules	create_schedule	route-specific
PATCH	/reports/schedules/{schedule_id}	update_schedule	route-specific
DELETE	/reports/schedules/{schedule_id}	delete_schedule	route-specific

system_errors

Method	Path	Function	Access hint
GET	/system-errors	list_system_errors	platform owner
PATCH	/system-errors/{error_id}/resolve	resolve_system_error	authenticated, platform owner
PATCH	/system-errors/{error_id}/reopen	reopen_system_error	authenticated, platform owner

WS

Method	Path	Function	Access hint
WEBSOCKET	/ws	websocket_auth_endpoint	route-specific

9. Business workflows



Marketplace request, unlock, offer and acceptance.



Loan disbursement, repayment, allocation and accounting.



LoanHub v1.0 Technical Architecture and Developer Handover



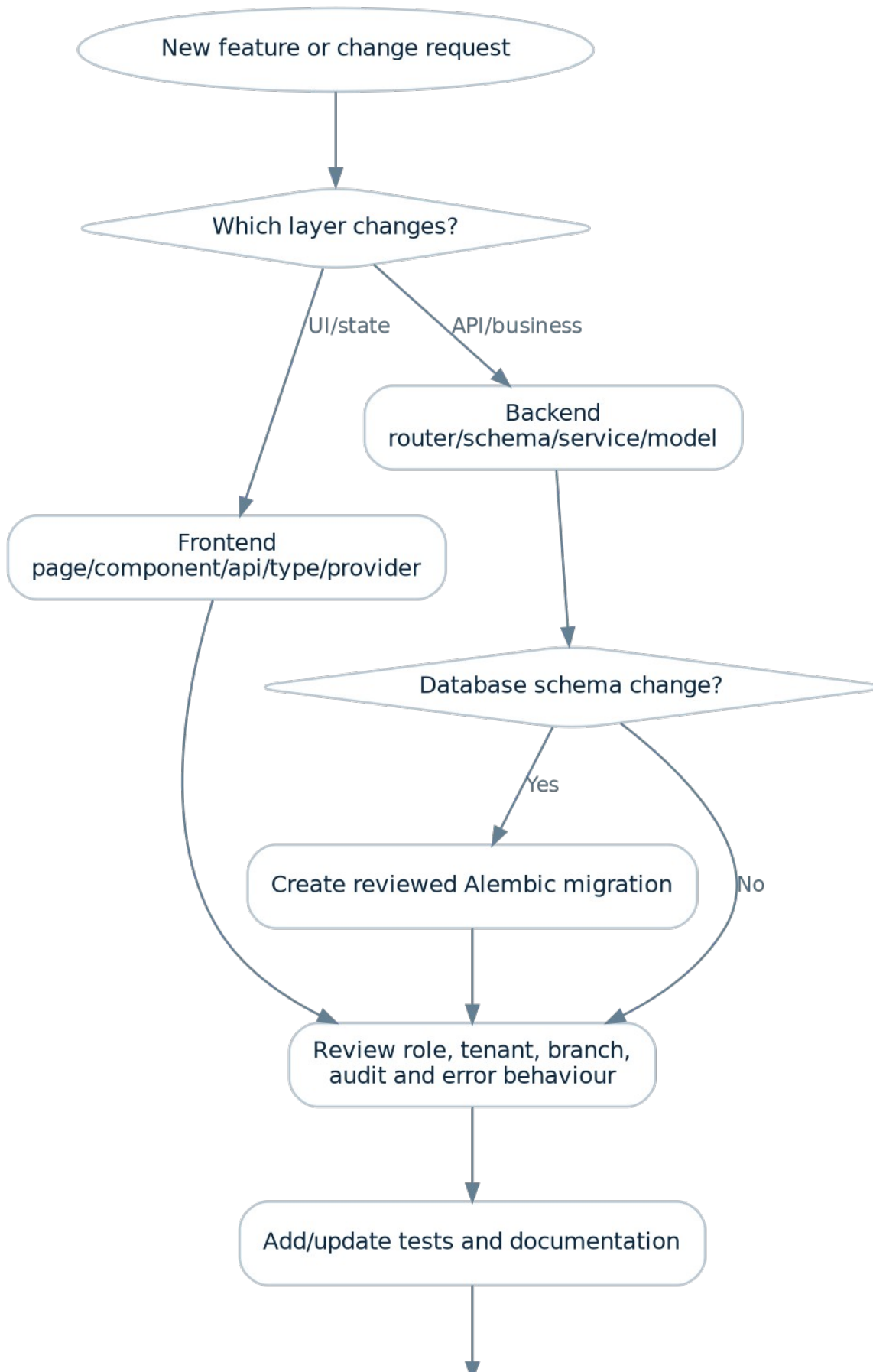
Business notification versus technical incident flow.



Chat and file security flow.



Midnight reconciliation and report generation.





10. Realtime chat and presence

A single RealtimeProvider owns one connection per browser tab. Before opening the socket, the frontend requests a short-lived HttpOnly WebSocket session cookie. The socket URL contains only the client ID. Chat and notification providers subscribe to the shared event stream, coalesce refreshes and deduplicate IDs.

``provider/realtimeProvider.tsx`` - protected connect and reconnect sequence

```
function scheduleReconnect() {
  if (disposedRef.current) {
    return;
  }

  const attempt =
    reconnectAttemptRef.current++;
  const delay = Math.min(
    30_000,
    1_000 * 2 ** attempt,
  );

  reconnectTimerRef.current =
    window.setTimeout(
      () => void connect(),
      delay,
    );
}

async function connect() {
  if (disposedRef.current) {
    return;
  }

  if (
    socketKeyRef.current === socketKey &&
    socketRef.current &&
    [
      WebSocket.CONNECTING,
      WebSocket.OPEN,
    ].includes(
      socketRef.current.readyState,
    )
  ) {
    return;
  }

  const currentGeneration = ++generation;
  clearReconnectTimer();
  closeCurrentSocket();

  try {
    // Creates a short-lived HttpOnly cookie scoped to /api/v1/ws.
    // The access token never appears in the WebSocket URL.
    await prepareWebSocketSession(
      activeCompanyId,
    );
  } catch {
    if (
      disposedRef.current ||
      currentGeneration !== generation
    ) {
      return;
    }

    setConnected(false);
    scheduleReconnect();
    return;
  }
}
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
if (
  disposedRef.current ||
  currentGeneration !== generation
) {
  return;
}

const socket = new WebSocket(
  websocketUrl(clientId),
  ["loanhub.v1"],
);

socketRef.current = socket;
socketKeyRef.current = socketKey;

socket.onopen = () => {
  if (
    socketRef.current !== socket ||
    disposedRef.current
  ) {
    socket.close();
    return;
  }

  reconnectAttemptRef.current = 0;
  setConnected(true);
};

socket.onmessage = (event) => {
  if (event.data === "pong") {
```

`routers/ws.py` - cookie/token compatibility and connection lifecycle

```
await manager.send_to_user(str(recipient_id), payload)

def _websocket_token(websocket: WebSocket) -> tuple[str | None, str | None]:
    # ASGI exposes parsed subprotocols directly. Keep the raw-header fallback
    # for older servers and retain the legacy query token temporarily.
    requested_protocols = list(
        websocket.scope.get("subprotocols") or []
    )
    if not requested_protocols:
        protocol_header = websocket.headers.get(
            "sec-websocket-protocol",
            ""
        )
        requested_protocols = [
            item.strip()
            for item in protocol_header.split(",")
            if item.strip()
        ]

    protocol_token = next(
        (
            item.removeprefix("loanhub.jwt.")
            for item in requested_protocols
            if item.startswith("loanhub.jwt.")
        ),
        None,
    )

    accepted_protocol = (
        "loanhub.v1"
        if "loanhub.v1" in requested_protocols
        else None
    )

    cookie_token = websocket.cookies.get(WS_SESSION_COOKIE)
    query_token = websocket.query_params.get("token")

    return (
        cookie_token or protocol_token or query_token,
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
        accepted_protocol,
    )

def _client_id(websocket: WebSocket, user_id: UUID) -> str:
    supplied = (websocket.query_params.get("client_id") or "").strip()
    if supplied and len(supplied) <= 128:
        return supplied
    return f"legacy-{user_id}"

@router.websocket('')
async def websocket_auth_endpoint(
    websocket: WebSocket,
    db: Session = Depends(get_db),
) -> None:
    token, accepted_protocol = _websocket_token(websocket)
    query_company_id = _uuid(websocket.query_params.get('company_id'))

    if not token:
        await websocket.close(code=1008, reason='Missing access token')
        return
    try:
        payload = decode_token(token)
    except HTTPException:
        await websocket.close(code=1008, reason='Invalid access token')
        return
    token_type = payload.get('token_type')
    if token_type not in {'access', 'websocket'}:
        await websocket.close(code=1008, reason='WebSocket authentication required')
        return

    payload_company_id = _uuid(str(payload.get('company_id') or ''))
    if (
        query_company_id
        and payload_company_id
        and query_company_id != payload_company_id
    ):
        await websocket.close(code=1008, reason='Company scope mismatch')
        return
    selected_company_id = query_company_id or payload_company_id

    user_id = _uuid(str(payload.get('user_id') or ''))
    if not user_id:
        await websocket.close(code=1008, reason='Invalid user')
        return

    user = (
        db.query(User)
        .options(selectinload(User.company_staff))
        .filter(User.id == user_id)
        .first()
    )
    if not user or not user.is_active:
        await websocket.close(code=1008, reason='Account unavailable')
        return

    memberships = [item for item in user.company_staff if item.is_active]
    if selected_company_id:
        memberships = [item for item in memberships if item.company_id == selected_company_id]
        if not memberships and user.role != UserRole.SUPERADMIN:
            await websocket.close(code=1008, reason='Company access denied')
            return

    channels: set[str] = {f'user-{user.id}'}
    if user.role == UserRole.SUPERADMIN:
        channels.add('superadmin')
    elif user.role == UserRole.BORROWER:
        channels.add(f'borrower-{user.id}')
    for membership in memberships:
        channels.add(f'company-{membership.company_id}')
        if membership.branch_id:
            channels.add(f'branch-{membership.branch_id}')
        if membership.role == UserRole.LOAN_OFFICER:
            channels.add(f'loan-officer-{user.id}')
```




```

client_id = _client_id(websocket, user.id)

await websocket.accept(subprotocol=accepted_protocol)
await manager.register_client(str(user.id), client_id, websocket)

for channel in channels:
    manager.subscribe(channel, websocket)

connected_at = datetime.now(timezone.utc)

```

- Do not open a second WebSocket in ChatProvider or NotificationProvider.
- Presence updates must not create actionable CRUD notifications.
- Use Redis for multi-worker event propagation.
- Deduplicate message and notification IDs after reconnect.
- Never place the full access JWT in the WebSocket URL.

11. Managed files and encryption

`services/file\_service.py` - filename, MIME, validation, atomic write and upload

```

}

def storage_root() -> Path:
    root = Path(settings.FILE_STORAGE_PATH).expanduser()
    if not root.is_absolute():
        root = Path(__file__).resolve().parents[1] / root
    root.mkdir(parents=True, exist_ok=True)
    return root.resolve()

def safe_filename(value: str) -> str:
    cleaned = re.sub(r'^A-Za-z0-9._-]+', '_', Path(value).name).strip()
    return cleaned[:240] or 'document'

def file_reference() -> str:
    return f'FILE-{uuid.uuid4().hex[:12].upper()}'

def _detect_mime(content: bytes, declared: str, extension: str | None) -> str:
    head = content[:32]
    if head.startswith(b'%PDF-'):
        return 'application/pdf'
    if head.startswith(b'\xff\xd8\xff'):
        return 'image/jpeg'
    if head.startswith(b'\x89PNG\r\n\x1a\n'):
        return 'image/png'
    if len(head) >= 12 and head[:4] == b'RIFF' and head[8:12] == b'WEBP':
        return 'image/webp'
    if head.startswith(b'OggS'):
        return 'audio/ogg'
    if head.startswith(b'\x1aE\xdf\xa3'):
        return 'audio/webm'
    if len(head) >= 12 and head[:4] == b'RIFF' and head[8:12] == b'WAVE':
        return 'audio/wav'
    if head.startswith(b'ID3') or (len(head) > 1 and head[0] == 0xFF and head[1] & 0xE0 == 0xE0):
        return 'audio/mp3'
    if len(head) >= 12 and b'ftyp' in head[4:12] and declared.startswith('audio/'):
        return 'audio/mp4'
    if head.startswith(b'PK\x03\x04') and declared in OFFICE_ZIP_TYPES:
        return declared
    if head.startswith(b'\xd0\xcf\x11\xe0') and declared in {
        'application/msword',
        'application/vnd.ms-excel',
    }:
        return declared

```



LoanHub v1.0 Technical Architecture and Developer Handover

```
if declared in {'text/plain', 'text/csv'}:
    try:
        content[:8192].decode('utf-8')
        return declared
    except UnicodeDecodeError as error:
        raise HTTPException(
            status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail='Text files must use UTF-8 encoding',
        ) from error
if extension in {'docx', 'xlsx'} and head.startswith(b'PK\x03\x04'):
    return declared
raise HTTPException(
    status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
    detail='The file content does not match an allowed document, image, PDF or audio format',
)

def _validate_content(
    content: bytes,
    *,
    original_name: str,
    declared_mime: str,
) -> tuple[str, str | None]:
    if not content:
        raise HTTPException(status_code=400, detail='The uploaded file is empty')

    extension = Path(original_name).suffix.lower().lstrip('.') or None
    if extension in BLOCKED_EXTENSIONS:
        raise HTTPException(
            status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail='Executable or active-content files are not allowed',
        )

    detected = _detect_mime(content, declared_mime, extension)
    if detected not in ALLOWED_MIME_TYPES:
        raise HTTPException(
            status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail='This file type is not allowed',
        )
    return detected, extension

def _atomic_write(destination: Path, content: bytes) -> None:
    destination.parent.mkdir(parents=True, exist_ok=True)
    temporary = destination.with_name(f'.{destination.name}.{uuid.uuid4().hex}.tmp')
    try:
        with temporary.open('wb') as output:
            output.write(content)
            output.flush()
            os.fsync(output.fileno())
        temporary.replace(destination)
    finally:
        temporary.unlink(missing_ok=True)

async def save_upload(
    db: Session,
    upload: UploadFile,
    context: TenantContext,
    *,
    category: str = 'general',
    visibility: str = 'private',
    description: str | None = None,
    linked_entity_type: str | None = None,
    linked_entity_id: str | None = None,
    is_confidential: bool = False,
    company_id=None,
    branch_id=None,
) -> ManagedFile:
    declared_mime = (upload.content_type or 'application/octet-stream').lower()
    original_name = safe_filename(upload.filename or 'document')

    maximum_mb = (
        settings.FILE_AUDIO_MAX_UPLOAD_MB
        if declared_mime.startswith('audio/')
        else settings.FILE_MAX_UPLOAD_MB
    )
```



```
)
maximum = max(1, maximum_mb) * 1024 * 1024
content = bytearray()

try:
    while True:
        chunk = await upload.read(1024 * 1024)
        if not chunk:
            break
        content.extend(chunk)
        if len(content) > maximum:
            raise HTTPException(
                status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
                detail=f'File exceeds the {maximum_mb} MB limit',
            )
finally:
    await upload.close()

plaintext = bytes(content)
detected_mime, extension = _validate_content(
    plaintext,
    original_name=original_name,
    declared_mime=declared_mime,
)

reference = file_reference()
stored_name = f'{uuid.uuid4().hex}.bin' if settings.FILE_ENCRYPTION_ENABLED else (
    f'{uuid.uuid4().hex}{("." + extension) if extension else ""}'
)
relative_key = Path(str(company_id or context.company_id or context.user_id)) / stored_name
destination = storage_root() / relative_key

payload = plaintext
encryption_nonce = None
encryption_version = None
if settings.FILE_ENCRYPTION_ENABLED:
    payload, encryption_nonce, encryption_version = encrypt_file_bytes(plaintext)

_atomic_write(destination, payload)

record = ManagedFile(
    owner_user_id=context.user.id,
    company_id=company_id if company_id is not None else context.company_id,
    branch_id=branch_id if branch_id is not None else context.branch_id,
    reference=reference,
    original_name=original_name,
    stored_name=stored_name,
    storage_key=str(relative_key),
    storage_provider='local',
    mime_type=detected_mime,
    detected_mime_type=detected_mime,
    extension=extension,
    size_bytes=len(plaintext),
    checksum_sha256=hashlib.sha256(plaintext).hexdigest(),
    is_encrypted=settings.FILE_ENCRYPTION_ENABLED,
    encryption_nonce=encryption_nonce,
    encryption_version=encryption_version,
```

Frontend upload rule

Correct multipart upload pattern

```
const form = new FormData();
form.append("file", selectedFile);
form.append("client_message_id", crypto.randomUUID());

// Do not set Content-Type manually.
// Axios/browser adds multipart/form-data with the boundary.
await api.post(`/chat/conversations/${conversationId}/files`, form);
```



File architecture

The database stores metadata and permissions; the media volume stores encrypted bytes. Back up and restore both together. For multi-server deployment, replace the local volume with S3/MinIO-compatible object storage and signed downloads.

12. Notifications, audit and error monitoring

`core/error_monitoring.py` - safe response, fingerprint and platform-owner alert

```
class SystemErrorMonitoringMiddleware(BaseHTTPMiddleware):
    """Capture unhandled failures without exposing diagnostics to ordinary users.

    The affected user receives one generic response containing a request ID.
    Only active platform owners receive the technical incident notification.
    Recurring fingerprints are grouped and rate-limited to avoid inbox spam.
    """

    async def dispatch(self, request: Request, call_next):
        request_id = request.headers.get('X-Request-ID') or uuid.uuid4().hex
        try:
            response = await call_next(request)
            response.headers['X-Request-ID'] = request_id
            return response
        except Exception as error:
            stack = traceback.format_exc()
            fingerprint = hashlib.sha256(
                (
                    f'{request.method}:{request.url.path}:'
                    f'{type(error).__name__}:{str(error)[:500]}'
                ).encode()
            ).hexdigest()

            user_id = _uuid(current_user_id.get())
            requested_company_id = _uuid(request.headers.get('X-Company-ID'))
            realtime_payloads: list[dict] = []
            db = SessionLocal()
            try:
                company_id, branch_id = _trusted_tenant_scope(
                    db,
                    user_id,
                    requested_company_id,
                )
                log = db.query(SystemErrorLog).filter(
                    SystemErrorLog.fingerprint == fingerprint
                ).first()
                was_resolved = bool(log and log.is_resolved)
                is_new = log is None

                if log:
                    log.occurrence_count += 1
                    log.last_seen_at = datetime.utcnow()
                    log.request_id = request_id
                    log.user_id = user_id
                    log.company_id = company_id
                    log.branch_id = branch_id
                    log.message = str(error)[:4000]
                    log.stack_trace = stack[-20000:]
                    log.user_agent = request.headers.get('user-agent')
                    log.is_resolved = False
                    log.resolved_at = None
                    log.resolved_by_user_id = None
                    log.resolution_notes = None
                else:
                    log = SystemErrorLog(
                        request_id=request_id,
                        fingerprint=fingerprint,
                        user_id=user_id,
                        company_id=company_id,
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
        branch_id=branch_id,
        method=request.method,
        path=request.url.path,
        status_code=500,
        error_type=type(error).__name__,
        message=str(error)[:4000] or 'Unhandled application error',
        stack_trace=stack[-20000:],
        severity='critical',
        environment=settings.ENVIRONMENT,
        user_agent=request.headers.get('user-agent'),
        context={
            'query': dict(request.query_params),
            'client': request.client.host if request.client else None,
        },
    )
    db.add(log)
    db.flush()

db.add(AuditLog(
    user_id=user_id,
    company_id=company_id,
    branch_id=branch_id,
    action='created' if is_new else 'updated',
    table_name='system_error_logs',
    entity_type='system_error_logs',
    record_id=log.id,
    description=(
        'A new system error was captured.'
        if is_new
        else 'A recurring system error was captured.'
    ),
    actor_role=None,
    severity='critical',
    status='failure',
    before_data={},
    after_data={
        'occurrence_count': log.occurrence_count,
        'is_resolved': False,
        'path': log.path,
        'error_type': log.error_type,
    },
    changed_fields=[] if is_new else ['occurrence_count', 'last_seen_at', 'is_resolved'],
    event_data={
        'automatic': True,
        'request_id': request_id,
        'fingerprint': fingerprint,
    },
    request_id=request_id,
    ip_address=request.client.host if request.client else None,
    user_agent=request.headers.get('user-agent'),
))

recent_cutoff = datetime.utcnow() - timedelta(minutes=15)
recent_notification_exists = db.query(Notification.id).filter(
    Notification.entity_type == 'system_error_logs',
    Notification.entity_id == str(log.id),
    Notification.created_at >= recent_cutoff,
).first() is not None

if is_new or was_resolved or not recent_notification_exists:
    platform_owners = db.query(User.id).filter(
        User.role == UserRole.SUPERADMIN,
        User.is_active.is_(True),
    ).all()
    for (admin_id,) in platform_owners:
        notification = Notification(
            user_id=admin_id,
            actor_user_id=user_id,
            company_id=company_id,
            branch_id=branch_id,
            title='System error detected',
            message=(
                f'{request.method} {request.url.path} failed. '
                f'Request reference: {request_id}.'
            ),
            notification_type=NotificationType.SYSTEM,
```



```

        entity_type='system_error_logs',
        entity_id=str(log.id),
        action_url='/superadmin/system-errors',
        action_label='Investigate error',
        icon='triangle-alert',
        priority='critical',
        data={
            'request_id': request_id,
            'fingerprint': fingerprint,
            'error_type': log.error_type,
            'occurrence_count': log.occurrence_count,
        },
    )
    db.add(notification)
    db.flush()
    realtime_payloads.append({
        'type': 'NOTIFICATION_CREATED',
        'user_id': str(admin_id),
        'notification': {
            'id': str(notification.id),
            'title': notification.title,
            'message': notification.message,
            'notification_type': notification.notification_type.value,
            'action_url': notification.action_url,
            'action_label': notification.action_label,
            'icon': notification.icon,
            'priority': notification.priority,
            'is_read': False,
            'created_at': notification.created_at.isoformat() if notification.created_at else
datetime.utcnow().isoformat(),
        },
    })

    db.commit()
except Exception:
    db.rollback()
finally:
    db.close()

```

The transparency listeners write normal create/update/delete audit events. Notification creation should remain actionable and role-targeted. Technical presence fields such as `last\_seen\_at` and routine timestamps must not produce user notifications.

13. Accounting and reporting

`services/accounting\_service.py` - idempotent chart bootstrap and advisory lock

```

("4200", "Transaction Fee Revenue", "revenue", "credit"),
("5000", "Payment Provider Expense", "expense", "debit"),
("5100", "Refund and Reversal Expense", "expense", "debit"),
]

def scope_key(company_id=None) -> tuple[str, str]:
    if company_id:
        return f"company:{company_id}", "company"
    return "platform", "platform"

def _chart_lock_id(scope: str) -> int:
    """Return a stable signed 64-bit PostgreSQL advisory-lock key."""
    digest = hashlib.sha256(f"loanhub:accounting-chart:{scope}".encode()).digest()
    value = int.from_bytes(digest[:8], byteorder="big", signed=False)
    return value if value < (1 << 63) else value - (1 << 64)

def ensure_chart(db: Session, *, company_id=None) -> list[AccountingAccount]:
    """Create any missing system accounts exactly once for a ledger scope.

```



Accounting pages may mount twice in React development mode and multiple API workers can receive the same bootstrap request concurrently. The transaction-scoped advisory lock serialises chart creation for PostgreSQL, while the unique constraint remains the final database safeguard.

```
"""
key, scope_type = scope_key(company_id)

bind = db.get_bind()
if bind is not None and bind.dialect.name == "postgresql":
    db.execute(select(func.pg_advisory_xact_lock(_chart_lock_id(key))))

existing = (
    db.query(AccountingAccount)
    .filter(AccountingAccount.scope_key == key)
    .order_by(AccountingAccount.code.asc())
    .all()
)
existing_codes = {account.code for account in existing}

rows = COMPANY_CHART if company_id else PLATFORM_CHART
for code, name, account_type, normal_balance in rows:
    if code in existing_codes:
        continue

    db.add(
        AccountingAccount(
            scope_key=key,
            scope_type=scope_type,
            company_id=company_id,
            code=code,
            name=name,
            account_type=account_type,
            normal_balance=normal_balance,
            is_system=True,
            is_active=True,
        )
    )

db.flush()

return (
    db.query(AccountingAccount)
    .filter(AccountingAccount.scope_key == key)
    .order_by(AccountingAccount.code.asc())
    .all()
)

def account_by_code(db: Session, key: str, code: str) -> AccountingAccount:
    account = db.query(AccountingAccount).filter(
        AccountingAccount.scope_key == key,
```

- The ledger scope is platform or one company.
- Journal entries require balanced debit and credit totals.
- Posted entries should be corrected through new entries, not hidden edits.
- Successful payment events can create mapped accounting entries.
- A qualified accountant should approve chart mapping, tax treatment and statutory reporting.

Reports

ReportingService measures tenants, staff, requests, offers, loans, balances, overdue, payments, performance and accounting metrics. Generated PDF/CSV files are stored through the same managed-file service.



14. Midnight automation

`services/nightly\_service.py` - due frequencies, idempotency and report generation

```
from __future__ import annotations

import logging
from datetime import date, timedelta
from zoneinfo import ZoneInfo

from sqlalchemy import text
from sqlalchemy.orm import Session

from database.config.config import settings
from database.models.branch import CompanyBranch
from database.models.company import LoanCompany
from database.models.enums import CompanyStatus
from database.models.reporting import GeneratedReport
from services.reporting_service import generate_report, period_for_frequency

logger = logging.getLogger('loanhub.nightly')
NIGHTLY_LOCK_ID = 62106420260720

def _frequency_due(local_date: date) -> list[str]:
    values = ['daily']
    if local_date.weekday() == 0:
        values.append('weekly')
    if local_date.day == 1:
        values.append('monthly')
    if local_date.month == 1 and local_date.day == 1:
        values.append('annual')
    return values

def _already_generated(
    db: Session,
    *,
    report_type: str,
    output_format: str,
    scope_type: str,
    company_id,
    branch_id,
    period_start: date,
    period_end: date,
) -> bool:
    query = db.query(GeneratedReport.id).filter(
        GeneratedReport.report_type == report_type,
        GeneratedReport.output_format == output_format,
        GeneratedReport.scope_type == scope_type,
        GeneratedReport.period_start == period_start,
        GeneratedReport.period_end == period_end,
    )
    query = query.filter(
        GeneratedReport.company_id.is_(None)
        if company_id is None
        else GeneratedReport.company_id == company_id,
        GeneratedReport.branch_id.is_(None)
        if branch_id is None
        else GeneratedReport.branch_id == branch_id,
    )
    return query.first() is not None

def run_midnight_reconciliation(db: Session, local_date: date | None = None) -> dict[str, int]:
    if not settings.MIDNIGHT_REPORTS_ENABLED:
        return {'generated': 0, 'skipped': 0}

    current_date = local_date or current_local_date()
    acquired = db.execute(
        text('SELECT pg_try_advisory_xact_lock(:lock_id)'),
        {'lock_id': NIGHTLY_LOCK_ID},
    )
```




LoanHub v1.0 Technical Architecture and Developer Handover

```
) .scalar()
if not acquired:
    return {'generated': 0, 'skipped': 1}

active_companies = db.query(LoanCompany).filter(
    LoanCompany.status == CompanyStatus.APPROVED,
    LoanCompany.is_active.is_(True),
).all()
active_branches = db.query(CompanyBranch).filter(
    CompanyBranch.is_active.is_(True),
).all()

scopes: list[tuple[str, object, object]] = [('platform', None, None)]
scopes.extend(('company', company.id, None) for company in active_companies)
scopes.extend(('branch', branch.company_id, branch.id) for branch in active_branches)

generated = 0
skipped = 0
for frequency in _frequency_due(current_date):
    period_start, period_end = period_for_frequency(frequency, current_date)
    for scope_type, company_id, branch_id in scopes:
        for output_format in settings.midnight_report_formats:
            report_type = f'{frequency}_reconciliation'
            if _already_generated(
                db,
                report_type=report_type,
                output_format=output_format,
                scope_type=scope_type,
                company_id=company_id,
                branch_id=branch_id,
                period_start=period_start,
                period_end=period_end,
            ):
                skipped += 1
                continue
            generate_report(
                db,
                report_type=report_type,
                output_format=output_format,
                scope_type=scope_type,
                period_start=period_start,
                period_end=period_end,
                company_id=company_id,
                branch_id=branch_id,
                generated_by_user_id=None,
            )
            generated += 1

db.commit()
logger.info(
    'Midnight reconciliation finished for %s: generated=%s skipped=%s',
    current_date,
    generated,
    skipped,
)
return {'generated': generated, 'skipped': skipped}

def current_local_date() -> date:
    from datetime import datetime
    return datetime.now(ZoneInfo(settings.APP_TIMEZONE)).date()
```

- Use `Africa/Maseru`, not the container default timezone.
- Acquire a PostgreSQL advisory lock so only one worker runs the midnight job.
- Use generated-report uniqueness/idempotency to avoid duplicates after restart.
- Keep the maintenance container running and monitor its logs.



15. Docker, CI/CD and Hostinger deployment



Figure 5. GitHub validation and production deployment pipeline.

`compose.yaml` - complete stack

```

name: loanhub

x-backend-environment: &backend-environment
  ENVIRONMENT: production
  DB_HOST: db
  DB_PORT: 5432
  DB_USER: ${DB_USER}
  DB_PASSWORD: ${DB_PASSWORD}
  DB_NAME: ${DB_NAME}
  SECRET_KEY: ${SECRET_KEY}
  FERNET_SECRET_KEY: ${FERNET_SECRET_KEY}
  CHAT_ENCRYPTION_KEY: ${CHAT_ENCRYPTION_KEY}
  FILE_ENCRYPTION_KEY: ${FILE_ENCRYPTION_KEY}
  FILE_ENCRYPTION_ENABLED: "true"
  JWT_PRIVATE_KEY_PATH: /run/host-secrets/jwt_private.pem
  JWT_PUBLIC_KEY_PATH: /run/host-secrets/jwt_public.pem
  REDIS_URL: redis://redis:6379/0
  CORS_ORIGINS: https://${APP_DOMAIN}
  PUBLIC_APP_URL: https://${APP_DOMAIN}
  FILE_STORAGE_PATH: /app/media/uploads
  APP_TIMEZONE: Africa/Maseru
  MIDNIGHT_REPORTS_ENABLED: "true"
  MIDNIGHT_REPORT_FORMATS: pdf,csv
  ENABLE_ROLE_IMPERSONATION: "true"
  PAYMENT MOCK MODE: ${PAYMENT MOCK MODE:-false}
  PAYMENT_CALLBACK_SECRET: ${PAYMENT_CALLBACK_SECRET}

services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      TZ: Africa/Maseru
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 12
    networks: [loanhub]
    security_opt: ["no-new-privileges:true"]

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    command: ["redis-server", "--appendonly", "yes", "--maxmemory-policy", "allkeys-lru"]
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 12
    networks: [loanhub]
    security_opt: ["no-new-privileges:true"]

  migrate:
    build:
      context: ./apps/backend

```



LoanHub v1.0 Technical Architecture and Developer Handover

```
image: loanhub-backend:${RELEASE_TAG:-local}
restart: "no"
environment: *backend-environment
command: ["alembic", "upgrade", "head"]
depends_on:
  db:
    condition: service_healthy
volumes:
  - ./secrets:/run/host-secrets:ro
networks: [loanhub]
security_opt: ["no-new-privileges:true"]

api:
  build:
    context: ./apps/backend
  image: loanhub-backend:${RELEASE_TAG:-local}
  restart: unless-stopped
  environment:
    <<: *backend-environment
    RUN_MIGRATIONS: "false"
    WEB_CONCURRENCY: ${WEB_CONCURRENCY:-2}
  depends_on:
    migrate:
      condition: service_completed_successfully
    redis:
      condition: service_healthy
  volumes:
    - ./secrets:/run/host-secrets:ro
    - media_data:/app/media
  expose: ["8000"]
  networks: [loanhub]
  init: true
  security_opt: ["no-new-privileges:true"]

maintenance:
  image: loanhub-backend:${RELEASE_TAG:-local}
  restart: unless-stopped
  environment: *backend-environment
  command: ["python", "docker/maintenance.py"]
  depends_on:
    api:
      condition: service_healthy
  volumes:
    - ./secrets:/run/host-secrets:ro
    - media_data:/app/media
  networks: [loanhub]
  init: true
  security_opt: ["no-new-privileges:true"]

frontend:
  build:
    context: ./apps/frontend
  args:
    NEXT_PUBLIC_API_URL: /api/v1
  image: loanhub-frontend:${RELEASE_TAG:-local}
  restart: unless-stopped
  depends_on:
    api:
      condition: service_healthy
  expose: ["3000"]
  networks: [loanhub]
  init: true
  security_opt: ["no-new-privileges:true"]

caddy:
  image: caddy:2-alpine
  restart: unless-stopped
  environment:
    APP_DOMAIN: ${APP_DOMAIN}
    TLS_EMAIL: ${TLS_EMAIL}
  depends_on:
    frontend:
      condition: service_healthy
  ports:
    - "80:80"
    - "443:443"
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
- "443:443/udp"
volumes:
  - ./infra/caddy/Caddyfile:/etc/caddy/Caddyfile:ro
  - caddy_data:/data
  - caddy_config:/config
networks: [loanhub]
security_opt: ["no-new-privileges:true"]

networks:
  loanhub:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  media_data:
  caddy_data:
  caddy_config:
```

`.github/workflows/deploy-hostinger.yml` - validation and SSH deployment

```
name: Deploy LoanHub to Hostinger VPS

on:
  push:
    branches: [main]
  workflow_dispatch:

concurrency:
  group: loanhub-production
  cancel-in-progress: false

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: pip
          cache-dependency-path: apps/backend/requirements.txt
      - run: pip install -r apps/backend/requirements.txt
      - name: Backend compile and migration graph
        working-directory: apps/backend
        env:
          SECRET_KEY: validation-secret
          FERNET_SECRET_KEY: validation-fernet
          CHAT_ENCRYPTION_KEY: validation-chat
          FILE_ENCRYPTION_KEY: validation-file
          JWT_PRIVATE_KEY_PATH: ${GITHUB_WORKSPACE}/.github/test-secrets/jwt_private.pem
          JWT_PUBLIC_KEY_PATH: ${GITHUB_WORKSPACE}/.github/test-secrets/jwt_public.pem
        run: |
          mkdir -p ${GITHUB_WORKSPACE}/.github/test-secrets
          openssl genpkey -algorithm RSA -out "$JWT_PRIVATE_KEY_PATH" -pkeyopt rsa_keygen_bits:2048
          openssl rsa -pubout -in "$JWT_PRIVATE_KEY_PATH" -out "$JWT_PUBLIC_KEY_PATH"
          python -m compileall -q .
          python scripts/validate_release.py
          test "$(alembic heads | wc -l)" -eq 1
          alembic upgrade head --sql >/tmp/upgrade.sql
          alembic downgrade fl9c4e7b620:base --sql >/tmp/downgrade.sql
      - uses: pnpm/action-setup@v4
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm
          cache-dependency-path: apps/frontend/pnpm-lock.yaml
      - name: Frontend typecheck and build
        working-directory: apps/frontend
        run: |
          pnpm install --frozen-lockfile
```



LoanHub v1.0 Technical Architecture and Developer Handover

```
pnpm typecheck
pnpm build
- name: Validate Compose
  run: docker compose config >/dev/null

deploy:
  needs: validate
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: actions/checkout@v4
    - name: Configure SSH
      env:
        SSH_PRIVATE_KEY: ${ secrets.HOSTINGER_SSH_PRIVATE_KEY }
        SSH_HOST: ${ secrets.HOSTINGER_HOST }
        SSH_PORT: ${ secrets.HOSTINGER_PORT }
      run: |
        install -m 700 -d ~/.ssh
        printf '%s\n' "$SSH_PRIVATE_KEY" > ~/.ssh/id_ed25519
        chmod 600 ~/.ssh/id_ed25519
        ssh-keyscan -p "${SSH_PORT:-22}" "${SSH_HOST}" >> ~/.ssh/known_hosts
    - name: Deploy committed release
      env:
        SSH_HOST: ${ secrets.HOSTINGER_HOST }
        SSH_PORT: ${ secrets.HOSTINGER_PORT }
        SSH_USER: ${ secrets.HOSTINGER_USER }
        DEPLOY_PATH: ${ secrets.HOSTINGER_DEPLOY_PATH }
      run: |
        ssh -p "${SSH_PORT:-22}" "${SSH_USER}@${SSH_HOST}" "cd '$DEPLOY_PATH' && ./scripts/deploy_hostinger.sh"
```

Deployment safeguards

- Keep database, Redis and internal app ports private; publish only Caddy 80/443.
- Run migrations as a one-shot service before starting the new API.
- Back up PostgreSQL and managed media before deployment.
- Store SSH, JWT and encryption secrets outside Git.
- Use health checks and retain the previous release for rollback.

16. Configuration and secrets

Variable	Example/default	Purpose
ENVIRONMENT	production	Runtime environment name
APP_TIMEZONE	Africa/Maseru	Business timezone for midnight tasks
PUBLIC_APP_URL	https://loanhub.example.com	Public browser URL
DB_HOST	db	PostgreSQL host
DB_PORT	5432	PostgreSQL port
DB_USER	loanhub	PostgreSQL user
DB_PASSWORD	CHANGE_ME	PostgreSQL password
DB_NAME	loanhub	PostgreSQL database
DATABASE_URL		Optional full SQLAlchemy URL



LoanHub v1.0 Technical Architecture and Developer Handover

Variable	Example/default	Purpose
CORS_ORIGINS	https://loanhub.example.com	Allowed browser origins
SECRET_KEY	CHANGE_ME_LONG_RANDOM_VALUE	Application secret
FERNET_SECRET_KEY	CHANGE_ME_URLSAFE_BASE64_KEY	General encryption key
CHAT_ENCRYPTION_KEY	CHANGE_ME_LONG_RANDOM_VALUE	Chat message encryption key
FILE_ENCRYPTION_KEY	CHANGE_ME_LONG_RANDOM_VALUE	Managed file encryption key
FILE_ENCRYPTION_ENABLED	true	Enable file-at-rest encryption
JWT_PRIVATE_KEY_PATH	/run/secrets/jwt_private.pem	RSA private key path
JWT_PUBLIC_KEY_PATH	/run/secrets/jwt_public.pem	RSA public key path
ACCESS_TOKEN_EXPIRE_MINUTES	30	Access-token lifetime
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh-token lifetime
ENABLE_ROLE_IMPERSONATION	true	Enable platform role switching
IMPERSONATION_MAX_MINUTES	30	Maximum impersonation session duration
REDIS_URL	redis://redis:6379/0	Redis connection URL
REDIS_REALTIME_CHANNEL	loanhub:realtime	Realtime pub/sub channel
FILE_STORAGE_PATH	/app/media/uploads	Managed media path
FILE_MAX_UPLOAD_MB	25	General upload limit
FILE_AUDIO_MAX_UPLOAD_MB	20	Voice-note upload limit
FILE_RETENTION_DAYS	3650	Retention setting
MAINTENANCE_INTERVAL_SECONDS	300	Worker polling interval
MIDNIGHT_REPORTS_ENABLED	true	Enable automatic midnight reports
MIDNIGHT_REPORT_FORMATS	pdf,csv	Generated report formats
PAYMENT MOCK_MODE	true	Use mock payment adapter
PAYMENT_CALLBACK_SECRET	CHANGE_ME	Provider callback secret
MARKETPLACE_DEFAULT_UNLOCK_FEE	25.00	Default request unlock fee
PRODUCT_NAME	LoanHub	Product display name
DEVELOPER_NAME	Ithute Solutions	Developer display name
LOANHUB_LOGO_PATH	assets/loanhub-horizontal-logo.png	Provider or runtime configuration
LOANHUB_APP_ICON_PATH	assets/loanhub-app-icon.png	Provider or runtime configuration
DEVELOPER_LOGO_PATH	assets/ithute-solutions-developer-logo.png	Provider or runtime configuration



LoanHub v1.0 Technical Architecture and Developer Handover

Variable	Example/default	Purpose
MPESA_API_KEY		Provider or runtime configuration
MPESA_PUBLIC_KEY		Provider or runtime configuration
MPESA_BASE_URL		Provider or runtime configuration
MPESA_MARKET	LS	Provider or runtime configuration
MPESA_COUNTRY	LS	Provider or runtime configuration
MPESA_CURRENCY	LSL	Provider or runtime configuration
MPESA_SHORTCODE		Provider or runtime configuration
MPESA_MODEL		Provider or runtime configuration
ECOCASH_API_KEY		Provider or runtime configuration
ECOCASH_API_SECRET		Provider or runtime configuration
ECOCASH_BASE_URL		Provider or runtime configuration
ECOCASH_MERCHANT_CODE		Provider or runtime configuration
ECOCASH_CALLBACK_URL		Provider or runtime configuration

Secret rule

Never commit `.env`, JWT private keys, provider credentials, database passwords or encryption keys. Rotate exposed credentials and reissue sessions after any leak.



17. Code-change playbook

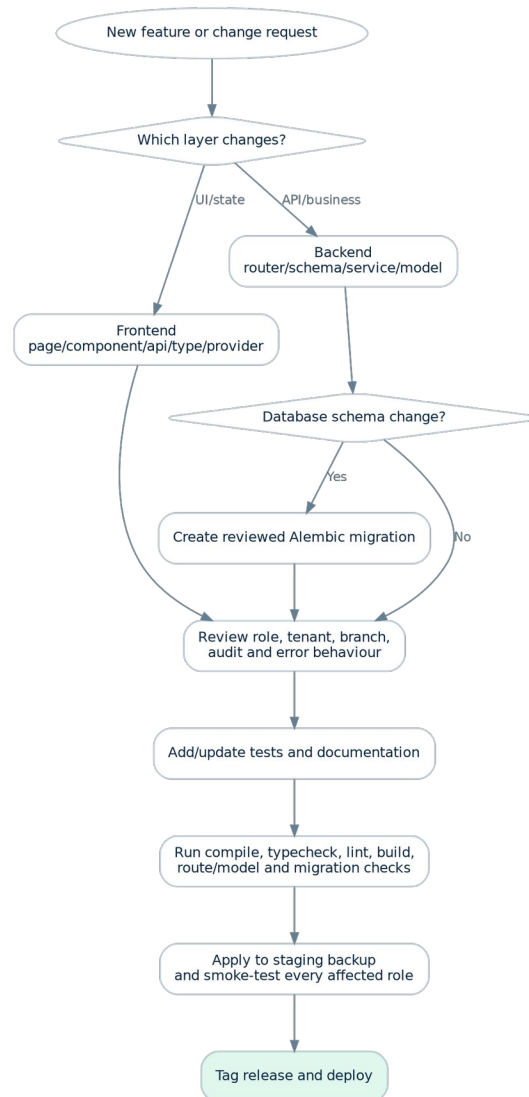


Figure 6. Safe decision flow for future changes.

Typical feature files

Requested change	Files/functions to inspect
Add or alter a page	`app/.../page.tsx`, related component, API client and TypeScript types
Add an endpoint	router decorator/function, schema, service and access-control dependency
Add a field/table	SQLAlchemy model, Pydantic schemas, service, migration, frontend type/form
Change tenant permissions	`core/access_control.py` plus every affected query
Change realtime behaviour	shared RealtimeProvider, WebSocket manager/router, Redis events



LoanHub v1.0 Technical Architecture and Developer Handover

Requested change	Files/functions to inspect
Change upload handling	Axios FormData handling, chat/files API, `file_service.py`
Change accounting	accounting router/schema/service/models and payment hooks
Change reports	report schemas/router/service/nightly schedule and file storage
Change deployment	Compose, Dockerfiles, Caddy, workflow and deployment script

Required questions before coding

1. Which roles can perform and view the action?
2. Which tenant and branch own the record?
3. Does retry require idempotency or a transaction lock?
4. Does the change require a migration?
5. Which notification and audit event should be created?
6. Does the feature upload/download files?
7. Which roles and failure cases must be tested?

18. Testing and release validation

Validation sequence

```
# Backend
python -m compileall -q .
python scripts/validate_release.py
alembic heads
alembic check
alembic upgrade head --sql > /tmp/upgrade.sql
alembic downgrade fla9c4e7b620:base --sql > /tmp/downgrade.sql

# Frontend
pnpm install --frozen-lockfile
pnpm typecheck
pnpm lint
pnpm build

# Stack
docker compose config
docker compose run --rm migrate
docker compose up -d
curl -f https://YOUR_DOMAIN/health
```

Role and security test matrix

Test type	Minimum coverage
Happy path	Creating, approving, paying, reporting and downloading as authorised roles



LoanHub v1.0 Technical Architecture and Developer Handover

Test type	Minimum coverage
Tenant isolation	Same ID requested by user from another company
Branch isolation	Branch-scoped role requesting another branch
Borrower privacy	Borrower sees only own records and authorised conversations
Retry/idempotency	Payment callback, report generation, accounting bootstrap, file/message client ID
Realtime	One socket per tab, reconnect, unread deduplication and presence
Files	Allowed/blocked types, large file, corrupted content, unauthorised download
Migration	Upgrade from production backup, current/head/check and rollback plan

19. Troubleshooting

Symptom	Root cause to inspect	Primary files
Login/auth 500	Migration mismatch, missing column, key/config or password library	auth router, user model, error log and Alembic current
Socket connection loop	Multiple providers or unstable dependency keys	RealtimeProvider and provider composition
WebSocket 403	Session-cookie handshake or hostname mismatch	auth websocket-session route, ws router and frontend API URL
Multipart 422	Forced application/json or missing form names	'lib/api.ts', chat/files APIs and router Form fields
Temporary bootstrap 500	Concurrent unique inserts	accounting service transaction lock
Notification spam	Presence/audit field classified as business event	transparency and notification service
Migration missing revision	Applied migration file removed	restore exact revision and maintain one chain
Migration deadlock	API/worker holds table locks	stop services, inspect PostgreSQL and use migrate service

20. Known boundaries and v1.1 backlog

- Official live M-Pesa/EcoCash onboarding, signed callbacks, enquiry and reversals.
- Automatic scheduled-report email delivery.
- S3/MinIO object storage and signed URLs for horizontal scaling.
- Enterprise malware scanning and quarantine workflow.
- Client-to-client end-to-end encryption.



LoanHub v1.0 Technical Architecture and Developer Handover

- Formal accounting/tax certification and period-lock workflows.
- Automated browser end-to-end coverage for all roles.

21. Appendix A - model details

accounting_accounts (AccountingAccount)

Chart of accounts.

Column	Type	PK	Nullable	Unique
scope_key	String		False	
scope_type	String		False	
company_id	UUID		True	
branch_id	UUID		True	
parent_id	UUID		True	
code	String		False	
name	String		False	
account_type	String		False	
normal_balance	String		False	
description	Text		True	
is_system	Boolean		False	
is_active	Boolean		False	

Foreign keys: company_id -> loan_companies.id, branch_id -> company_branches.id, parent_id -> accounting_accounts.id

Relationships: parent -> AccountingAccount

audit_logs (AuditLog)

Who changed what, where and when.

Column	Type	PK	Nullable	Unique
user_id	UUID		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
company_id	UUID		True	
branch_id	UUID		True	
action	String		False	
table_name	String		True	
entity_type	String		True	
record_id	UUID		True	
description	Text		True	
actor_role	String		True	
severity	String		False	
status	String		False	
before_data	JSONB		False	
after_data	JSONB		False	
changed_fields	JSONB		False	
event_data	JSONB		False	
request_id	String		True	
ip_address	String		True	
user_agent	String		True	
duration_ms	Integer		True	

Foreign keys: user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id

Relationships: user -> User, company -> LoanCompany, branch -> CompanyBranch

borrower_documents (BorrowerDocument)

Borrower identity and supporting evidence.



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
borrower_id	UUID		False	
document_type	Enum		False	
file_name	String		False	
file_url	String		False	
is_verified	Boolean			

Foreign keys: borrower_id -> borrowers.id

Relationships: borrower -> Borrower

borrowers (Borrower)

Borrower profile and affordability context.

Column	Type	PK	Nullable	Unique
user_id	UUID		False	Yes
employment_status	Enum		False	
employer_name	String		True	
job_title	String		True	
monthly_income	Numeric		True	
salary_date	String		True	
has_existing_loans	Boolean		False	
existing_loan_total	Numeric		False	
consent_to_share_profile	Boolean		False	
consent_to_credit_checks	Boolean		False	

Foreign keys: user_id -> users.id

Relationships: user -> User, loans -> ClientCompanyLoan, loan_requests -> LoanRequest, documents -> BorrowerDocument, payment_transactions -> PaymentTransaction



broadcast (Broadcast)

Domain persistence record.

Column	Type	PK	Nullable	Unique
user_id	UUID		True	
channel	String		False	
title	String		False	
message	Text		False	
event_type	String		False	
entity	String		True	
entity_id	String		True	
data	JSONB			
is_read	Boolean		False	

Foreign keys: user_id -> users.id

Relationships: user -> User

chat_conversations (ChatConversation)

Direct/group chat container.

Column	Type	PK	Nullable	Unique
reference	String		False	Yes
company_id	UUID		True	
branch_id	UUID		True	
created_by_user_id	UUID		True	
title	String		True	
conversation_type	String		False	
context_type	String		True	
context_id	String		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
is_group	Boolean		False	
is_archived	Boolean		False	
last_message_at	DateTime		True	

Foreign keys: company_id -> loan_companies.id, branch_id -> company_branches.id, created_by_user_id -> users.id

Relationships: created_by_user -> User, participants -> ChatParticipant, messages -> ChatMessage

chat_message_attachments (ChatMessageAttachment)

Links messages to managed files.

Column	Type	PK	Nullable	Unique
message_id	UUID		False	
file_id	UUID		False	
caption	String		True	

Foreign keys: message_id -> chat_messages.id, file_id -> managed_files.id

Relationships: message -> ChatMessage, file -> ManagedFile

chat_messages (ChatMessage)

Encrypted/managed chat message records.

Column	Type	PK	Nullable	Unique
conversation_id	UUID		False	
sender_user_id	UUID		True	
company_id	UUID		True	
branch_id	UUID		True	
reply_to_message_id	UUID		True	
message_type	String		False	
body	Text		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
body_ciphertext	Text		True	
body_nonce	String		True	
encryption_version	String		True	
client_message_id	String		False	
metadata_json	JSONB		False	
edited_at	DateTime		True	
deleted_at	DateTime		True	

Foreign keys: conversation_id -> chat_conversations.id, sender_user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id, reply_to_message_id -> chat_messages.id

Relationships: conversation -> ChatConversation, sender -> User, reply_to -> ChatMessage, attachments -> ChatMessageAttachment

chat_participants (ChatParticipant)

Conversation membership and read state.

Column	Type	PK	Nullable	Unique
conversation_id	UUID		False	
user_id	UUID		False	
participant_role	String		True	
is_admin	Boolean		False	
is_active	Boolean		False	
joined_at	DateTime		False	
last_read_at	DateTime		True	
muted_until	DateTime		True	

Foreign keys: conversation_id -> chat_conversations.id, user_id -> users.id

Relationships: conversation -> ChatConversation, user -> User



client_company_loan (ClientCompanyLoan)

Domain persistence record.

Column	Type	PK	Nullable	Unique
loan_request_id	UUID		False	
loan_offer_id	UUID		False	
company_id	UUID		False	
branch_id	UUID		True	
borrower_id	UUID		False	
loan_reference	String		False	Yes
principal_amount	Numeric		False	
interest_rate	Numeric		False	
processing_fee	Numeric		False	
total_repayable	Numeric		False	
repayment_type	Enum		False	
repayment_period	Integer		False	
installment_amount	Numeric		False	
approved_at	DateTime		True	
disbursed_at	DateTime		True	
first_payment_due	Date		True	
maturity_date	Date		True	
amount_paid	Numeric		False	
balance	Numeric		False	
status	Enum		False	
risk_level	Enum		False	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
is_overdue	Boolean		False	
approved_by_user_id	UUID		True	
disbursed_by_user_id	UUID		True	

Foreign keys: loan_request_id -> loan_requests.id, loan_offer_id -> loan_offers.id, company_id -> loan_companies.id, branch_id -> company_branches.id, borrower_id -> borrowers.id, approved_by_user_id -> users.id, disbursed_by_user_id -> users.id

Relationships: borrower -> Borrower, company -> LoanCompany, branch -> CompanyBranch, loan_request -> LoanRequest, loan_offer -> LoanOffer, approved_by -> User, disbursed_by -> User, installments -> RepaymentInstallment, payment_transactions -> PaymentTransaction

company_branches (CompanyBranch)

Domain persistence record.

Column	Type	PK	Nullable	Unique
company_id	UUID		False	
name	String		False	
district	String		False	
town	String		True	
address	Text		True	
phone	String		True	
email	String		True	
is_active	Boolean		False	

Foreign keys: company_id -> loan_companies.id

Relationships: company -> LoanCompany, staff_members -> CompanyStaff, loans -> ClientCompanyLoan

company_staff (CompanyStaff)

User membership, company role and branch scope.



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
user_id	UUID		False	
company_id	UUID		False	
branch_id	UUID		True	
role	Enum		False	
is_active	Boolean		False	

Foreign keys: user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id

Relationships: user -> User, company -> LoanCompany, branch -> CompanyBranch, employee_profile -> EmployeeProfile

company_subscriptions (CompanySubscription)

Company plan lifecycle.

Column	Type	PK	Nullable	Unique
company_id	UUID		False	
plan_id	UUID		True	
plan_name	String		False	
amount	Numeric		False	
start_date	Date		False	
end_date	Date		False	
billing_cycle	Enum		False	
status	Enum		False	
auto_renew	Boolean		False	
payment_provider	Enum		True	
external_reference	String		True	Yes

Foreign keys: company_id -> loan_companies.id, plan_id -> subscription_plans.id

Relationships: company -> LoanCompany, plan -> SubscriptionPlan



employee_profiles (EmployeeProfile)

Domain persistence record.

Column	Type	PK	Nullable	Unique
staff_id	UUID		False	
company_id	UUID		False	
branch_id	UUID		True	
reports_to_staff_id	UUID		True	
employee_number	String		False	
job_title	String		True	
department	String		True	
employment_type	String		False	
employment_status	String		False	
hire_date	Date		True	
probation_end_date	Date		True	
termination_date	Date		True	
base_salary	Numeric		True	
currency	String		False	
skills	JSONB		False	
target_config	JSONB		False	
notes	Text		True	
is_manager	Boolean		False	

Foreign keys: staff_id -> company_staff.id, company_id -> loan_companies.id, branch_id -> company_branches.id, reports_to_staff_id -> company_staff.id

Relationships: staff -> CompanyStaff, reports_to -> CompanyStaff, company -> LoanCompany, branch -> CompanyBranch, goals -> PerformanceGoal, reviews -> PerformanceReview



generated_reports (GeneratedReport)

Generated report metadata and managed-file link.

Column	Type	PK	Nullable	Unique
reference	String		False	Yes
scope_type	String		False	
company_id	UUID		True	
branch_id	UUID		True	
schedule_id	UUID		True	
generated_by_user_id	UUID		True	
file_id	UUID		True	
title	String		False	
report_type	String		False	
output_format	String		False	
period_start	Date		False	
period_end	Date		False	
status	String		False	
metrics	JSONB		False	
error_message	Text		True	
generated_at	DateTime		False	

Foreign keys: company_id -> loan_companies.id, branch_id -> company_branches.id, schedule_id -> report_schedules.id, generated_by_user_id -> users.id, file_id -> managed_files.id

Relationships: file -> ManagedFile, schedule -> ReportSchedule

journal_entries (JournalEntry)

Accounting transaction headers.



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
scope_key	String		False	
scope_type	String		False	
company_id	UUID		True	
branch_id	UUID		True	
created_by_user_id	UUID		True	
posted_by_user_id	UUID		True	
entry_number	String		False	
entry_date	Date		False	
description	Text		False	
reference_type	String		True	
reference_id	String		True	
status	String		False	
total_debit	Numeric		False	
total_credit	Numeric		False	
posted_at	DateTime		True	
voided_at	DateTime		True	

Foreign keys: company_id -> loan_companies.id, branch_id -> company_branches.id, created_by_user_id -> users.id, posted_by_user_id -> users.id

Relationships: lines -> JournalLine

journal_lines (JournalLine)

Balanced debit/credit lines.

Column	Type	PK	Nullable	Unique
journal_entry_id	UUID		False	
account_id	UUID		False	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
description	String		True	
debit	Numeric		False	
credit	Numeric		False	

Foreign keys: journal_entry_id -> journal_entries.id, account_id -> accounting_accounts.id

Relationships: entry -> JournalEntry, account -> AccountingAccount

lender_access_requests (LenderAccessRequest)

Legacy/controlled lender access request.

Column	Type	PK	Nullable	Unique
loan_request_id	UUID		False	
company_id	UUID		False	
requested_by_user_id	UUID		False	
status	Enum			
message	Text		True	

Foreign keys: loan_request_id -> loan_requests.id, company_id -> loan_companies.id, requested_by_user_id -> users.id

Relationships: loan_request -> LoanRequest, company -> LoanCompany, requested_by -> User

loan_companies (LoanCompany)

Domain persistence record.

Column	Type	PK	Nullable	Unique
name	String		False	Yes
registration_number	String		True	Yes
license_number	String		True	Yes
phone	String		False	
email	String		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
website	String		True	
address	Text		True	
district	String		True	
status	Enum		False	
is_active	Boolean		False	

Relationships: branches -> CompanyBranch, staff_members -> CompanyStaff, loan_products -> LoanProduct, offers -> LoanOffer, subscriptions -> CompanySubscription, marketplace_unlocks -> MarketplaceUnlock, payment_transactions -> PaymentTransaction, loans -> ClientCompanyLoan

loan_offers (LoanOffer)

Company offers made against requests.

Column	Type	PK	Nullable	Unique
loan_request_id	UUID		False	
company_id	UUID		False	
branch_id	UUID		True	
offered_by_user_id	UUID		False	
approved_amount	Numeric		False	
term_months	Integer		False	
interest_rate_percent	Numeric		False	
processing_fee	Numeric		False	
monthly_repayment	Numeric		False	
total_repayment	Numeric		False	
notes	Text		True	
status	Enum		False	
expires_at	DateTime		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
accepted_at	DateTime		True	
created_at	DateTime		False	

Foreign keys: loan_request_id -> loan_requests.id, company_id -> loan_companies.id, branch_id -> company_branches.id, offered_by_user_id -> users.id

Relationships: loan_request -> LoanRequest, company -> LoanCompany, branch -> CompanyBranch, offered_by -> User, loan -> ClientCompanyLoan

loan_products (LoanProduct)

Company-controlled loan products and pricing bounds.

Column	Type	PK	Nullable	Unique
company_id	UUID		False	
name	String		False	
description	Text		True	
min_amount	Numeric		False	
max_amount	Numeric		False	
min_term_months	Integer		False	
max_term_months	Integer		False	
interest_rate_percent	Numeric		False	
processing_fee	Numeric		False	
is_active	Boolean		False	

Foreign keys: company_id -> loan_companies.id

Relationships: company -> LoanCompany

loan_request_documents (LoanRequestDocument)

Domain persistence record.



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
loan_request_id	UUID		False	
document_type	Enum		False	
file_name	String		False	
file_url	String		False	

Foreign keys: loan_request_id -> loan_requests.id

Relationships: loan_request -> LoanRequest

loan_requests (LoanRequest)

Borrower marketplace funding requests.

Column	Type	PK	Nullable	Unique
borrower_id	UUID		False	
requested_amount	Numeric		False	
preferred_term_months	Integer		True	
loan_purpose	Text		True	
status	Enum		False	
visible_to_lenders	Boolean		False	
allow_lenders_to_call	Boolean		False	
selected_offer_id	UUID		True	Yes
submitted_at	DateTime		True	
expires_at	DateTime		True	
accepted_at	DateTime		True	
created_at	DateTime		False	

Foreign keys: borrower_id -> borrowers.id, selected_offer_id -> loan_offers.id



LoanHub v1.0 Technical Architecture and Developer Handover

Relationships: borrower -> Borrower, offers -> LoanOffer, selected_offer -> LoanOffer, documents -> LoanRequestDocument, access_requests -> LenderAccessRequest, marketplace_unlocks -> MarketplaceUnlock, payment_transactions -> PaymentTransaction, loan -> ClientCompanyLoan

managed_files (ManagedFile)

File metadata, visibility, integrity and encryption.

Column	Type	PK	Nullable	Unique
owner_user_id	UUID		True	
company_id	UUID		True	
branch_id	UUID		True	
reference	String		False	Yes
original_name	String		False	
stored_name	String		False	
storage_key	String		False	Yes
storage_provider	String		False	
mime_type	String		False	
detected_mime_type	String		True	
extension	String		True	
size_bytes	BigInteger		False	
checksum_sha256	String		False	
is_encrypted	Boolean		False	
encryption_nonce	String		True	
encryption_version	String		True	
scan_status	String		False	
quarantined_reason	Text		True	
category	String		False	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
visibility	String		False	
description	Text		True	
linked_entity_type	String		True	
linked_entity_id	String		True	
is_confidential	Boolean		False	
is_deleted	Boolean		False	
deleted_at	DateTime		True	

Foreign keys: owner_user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id

Relationships: owner -> User, company -> LoanCompany, branch -> CompanyBranch

marketplace_unlocks (MarketplaceUnlock)

Company access/unlock events for requests.

Column	Type	PK	Nullable	Unique
company_id	UUID		False	
loan_request_id	UUID		False	
payment_transaction_id	UUID		True	Yes
unlocked_by_user_id	UUID		True	
price_paid	Numeric		False	
status	Enum		False	
unlocked_at	DateTime		True	
expires_at	DateTime		True	

Foreign keys: company_id -> loan_companies.id, loan_request_id -> loan_requests.id, payment_transaction_id -> payment_transactions.id, unlocked_by_user_id -> users.id

Relationships: company -> LoanCompany, loan_request -> LoanRequest, payment_transaction -> PaymentTransaction, unlocked_by -> User



notifications (Notification)

Persistent user-targeted actionable events.

Column	Type	PK	Nullable	Unique
user_id	UUID		False	
actor_user_id	UUID		True	
company_id	UUID		True	
branch_id	UUID		True	
title	String		False	
message	Text		False	
notification_type	Enum		False	
event_type	String		False	
action	String		False	
entity_type	String		True	
entity_id	String		True	
action_url	String		True	
icon	String		True	
priority	String		False	
data	JSONB		False	
deduplication_key	String		True	
is_read	Boolean		False	
read_at	DateTime		True	
is_archived	Boolean		False	
archived_at	DateTime		True	
related_loan_request_id	UUID		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
related_offer_id	UUID		True	

Foreign keys: user_id -> users.id, actor_user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id, related_loan_request_id -> loan_requests.id, related_offer_id -> loan_offers.id

Relationships: recipient -> User, actor -> User, company -> LoanCompany, branch -> CompanyBranch

payment_allocations (PaymentAllocation)

Allocation of repayments to instalments.

Column	Type	PK	Nullable	Unique
payment_id	UUID		False	
installment_id	UUID		False	
amount	Numeric		False	

Foreign keys: payment_id -> payment_transactions.id, installment_id -> repayment_installments.id

Relationships: payment -> PaymentTransaction, installment -> RepaymentInstallment

payment_transactions (PaymentTransaction)

Provider-neutral inbound/outbound payment ledger.

Column	Type	PK	Nullable	Unique
company_id	UUID		True	
borrower_id	UUID		True	
loan_request_id	UUID		True	
loan_id	UUID		True	
initiated_by_user_id	UUID		True	
provider	Enum		False	
direction	Enum		False	
purpose	Enum		False	
status	Enum		False	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
amount	Numeric		False	
currency	String		False	
payer_phone	String		True	
payee_phone	String		True	
idempotency_key	String		False	Yes
provider_reference	String		True	Yes
provider_payload	JSONB		False	
failure_reason	Text		True	
completed_at	DateTime		True	

Foreign keys: company_id -> loan_companies.id, borrower_id -> borrowers.id, loan_request_id -> loan_requests.id, loan_id -> client_company_loan.id, initiated_by_user_id -> users.id

Relationships: company -> LoanCompany, borrower -> Borrower, loan_request -> LoanRequest, loan -> ClientCompanyLoan, initiated_by -> User, unlock -> MarketplaceUnlock, allocations -> PaymentAllocation

people (Person)

Reusable person identity/profile data.

Column	Type	PK	Nullable	Unique
user_id	UUID		False	Yes
first_name	String		False	
middle_name	String		True	
last_name	String		False	
gender	Enum		True	
date_of_birth	Date		True	
national_id	String		True	Yes
passport_number	String		True	Yes



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
marital_status	Enum		True	
nationality	String		True	
district	String		True	
town_or_village	String		True	
physical_address	Text		True	

Foreign keys: user_id -> users.id

Relationships: user -> User

performance_goals (PerformanceGoal)

Targets and progress.

Column	Type	PK	Nullable	Unique
employee_id	UUID		False	
company_id	UUID		False	
branch_id	UUID		True	
created_by_user_id	UUID		True	
title	String		False	
description	Text		True	
category	String		False	
target_value	Numeric		False	
current_value	Numeric		False	
unit	String		False	
weight	Numeric		False	
period_start	Date		False	
period_end	Date		False	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
status	String		False	
completed_at	DateTime		True	

Foreign keys: employee_id -> employee_profiles.id, company_id -> loan_companies.id, branch_id -> company_branches.id, created_by_user_id -> users.id

Relationships: employee -> EmployeeProfile, created_by_user -> User

performance_reviews (PerformanceReview)

Review-period assessment data.

Column	Type	PK	Nullable	Unique
employee_id	UUID		False	
reviewer_staff_id	UUID		True	
company_id	UUID		False	
branch_id	UUID		True	
period_start	Date		False	
period_end	Date		False	
overall_score	Numeric		False	
rating	String		False	
status	String		False	
strengths	Text		True	
improvements	Text		True	
comments	Text		True	
metrics	JSONB		False	
employee_acknowledged_at	DateTime		True	

Foreign keys: employee_id -> employee_profiles.id, reviewer_staff_id -> company_staff.id, company_id -> loan_companies.id, branch_id -> company_branches.id



LoanHub v1.0 Technical Architecture and Developer Handover

Relationships: employee -> EmployeeProfile, reviewer -> CompanyStaff

refresh_tokens (RefreshToken)

Refresh-token lifecycle and revocation.

Column	Type	PK	Nullable	Unique
jti	String		False	Yes
user_id	UUID		False	
revoked	Boolean		False	
expires_at	DateTime		False	

Foreign keys: user_id -> users.id

Relationships: user -> User

repayment_installments (RepaymentInstallment)

Scheduled instalments and due/paid status.

Column	Type	PK	Nullable	Unique
loan_id	UUID		False	
installment_number	Integer		False	
due_date	Date		False	
principal_due	Numeric		False	
interest_due	Numeric		False	
fee_due	Numeric		False	
total_due	Numeric		False	
paid_amount	Numeric		False	
status	Enum		False	
paid_at	DateTime		True	

Foreign keys: loan_id -> client_company_loan.id

Relationships: loan -> ClientCompanyLoan, allocations -> PaymentAllocation



report_schedules (ReportSchedule)

Recurring report configuration.

Column	Type	PK	Nullable	Unique
scope_type	String		False	
company_id	UUID		True	
branch_id	UUID		True	
created_by_user_id	UUID		True	
name	String		False	
report_type	String		False	
frequency	String		False	
output_format	String		False	
recipients	JSONB		False	
is_active	Boolean		False	
next_run_at	DateTime		False	
last_run_at	DateTime		True	
last_status	String		True	
last_error	Text		True	

Foreign keys: company_id -> loan_companies.id, branch_id -> company_branches.id, created_by_user_id -> users.id

subscription_plans (SubscriptionPlan)

Platform commercial plan configuration.

Column	Type	PK	Nullable	Unique
code	String		False	Yes
name	String		False	
description	String		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
monthly_price	Numeric		False	
annual_price	Numeric		False	
marketplace_unlock_fee	Numeric		False	
transaction_fee_percent	Numeric		False	
features	JSONB		False	
limits	JSONB		False	
is_active	Boolean		False	
is_public	Boolean		False	

Relationships: subscriptions -> CompanySubscription

system_error_logs (SystemErrorLog)

Grouped technical failures and resolution status.

Column	Type	PK	Nullable	Unique
request_id	String		True	
fingerprint	String		False	Yes
user_id	UUID		True	
company_id	UUID		True	
branch_id	UUID		True	
method	String		True	
path	String		False	
status_code	Integer		False	
error_type	String		False	
message	Text		False	
stack_trace	Text		True	



LoanHub v1.0 Technical Architecture and Developer Handover

Column	Type	PK	Nullable	Unique
severity	String		False	
environment	String		True	
user_agent	String		True	
context	JSONB		False	
occurrence_count	Integer		False	
first_seen_at	DateTime		False	
last_seen_at	DateTime		False	
is_resolved	Boolean		False	
resolved_at	DateTime		True	
resolved_by_user_id	UUID		True	
resolution_notes	Text		True	

Foreign keys: user_id -> users.id, company_id -> loan_companies.id, branch_id -> company_branches.id, resolved_by_user_id -> users.id

Relationships: user -> User, resolved_by -> User, company -> LoanCompany, branch -> CompanyBranch

users (User)

Authentication accounts, global role, status and presence.

Column	Type	PK	Nullable	Unique
email	String		True	Yes
phone	String		False	Yes
password_hash	String		False	
role	Enum		False	
is_active	Boolean		False	
is_verified	Boolean		False	
last_seen_at	DateTime		True	



Relationships: person -> Person, refresh_tokens -> RefreshToken, broadcast -> Broadcast, borrower_profile -> Borrower, company_staff -> CompanyStaff, notifications -> Notification

22. Appendix B - source catalogs

The accompanying documentation ZIP includes CSV files for API routes, models, frontend routes, migrations and environment variables, plus editable Graphviz sources, SVG/PNG diagrams and selected source files. These machine-readable catalogs should be regenerated whenever v1.0 is upgraded.